

AI-Powered Lost and Found System

PHASE I REPORT

Submitted by

SHYAAM KK	230701313
ROONEY BALA I	230701269
TARUN C	230701358
VERZAN	230701378

in partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

ANNA UNIVERSITY, CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report “**AI-POWERED LOST AND FOUND SYSTEM**” is the Bonafide work of **SHYAAM KK (230701313)**, **ROONEY BALA I (230701269)**, **TARUN C (230701358)**, and **VERZAN (230701378)** who carried out the project work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

This project addresses the following Sustainable Development Goal: **Goal No. 11: Sustainable Cities and Communities.**

SIGNATURE

Dr. J. Manoranjini
Professor and Head,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

SIGNATURE

Mrs. M. Shyni
SUPERVISOR
Assistant Professor,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner

External Examiner

ABSTRACT

The AI-Powered Intelligent Lost and Found System is a full-stack web application designed to simplify and automate the process of reporting, identifying, and recovering lost and found items. Conventional lost-and-found systems generally depend on manual searching and keyword-based descriptions, making it difficult to identify relevant matches when users provide different descriptions or when multiple similar items are reported. The proposed system addresses this limitation by integrating artificial intelligence-based image and text similarity analysis into a unified matching workflow.

The system allows users to register and securely authenticate, report lost or found items, provide item descriptions, specify locations and dates, and upload item images. For intelligent matching, a pre-trained CLIP model is used to generate image embeddings, while the all-MiniLM-L6-v2 model from the sentence-transformers library generates semantic text embeddings. Cosine similarity is then used to determine the similarity between lost and found reports. These signals are combined using a hybrid weighted scoring mechanism consisting of 45% image similarity, 35% text similarity, 10% location similarity, and 10% date similarity. The resulting candidates are ranked and presented to users along with an explainable breakdown of the individual similarity scores.

The application is implemented using React, Vite, TypeScript, Tailwind CSS, and React Router for the frontend, and Python with FastAPI and Pydantic for the backend. PostgreSQL through Supabase, along with pgvector, is used for structured data and cached AI embeddings, while Supabase Authentication and Storage support secure authentication and image management. Security is enforced through JWT verification, server-side authorization checks, image validation, environment-based secrets, and database-level Row-Level Security. By combining multimodal AI, explainable hybrid scoring, cached embeddings, and human verification, the proposed system provides a practical and trustworthy approach to improving the efficiency of lost-and-found item identification and recovery.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavour to put forth this report. Our sincere thanks to our Chairman **Mr. S. MEGANATHAN, B.E., F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavouring in educating us in their premier institution.

Our sincere thanks to **Dr. S. N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J. Manoranjini, Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work.

We convey our sincere and deepest gratitude to our internal guide and mentor, **Mrs.M.Shyni Assistant Professor**, Department of Computer Science and Engineering, for his valuable guidance throughout the course of the project. We thank our Project Coordinator, **Dr . Madhusudhanan .S**, Associate Professor, Department of Computer Science and Engineering, for the useful tips during our reviews that helped us to build our project.

SHYAAM KK	230701313
ROONEY BALA I	230701269
TARUN C	230701358
VERZAN	230701378

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iv
	LIST OF TABLES	viii
	LIST OF FIGURES	ix
	LIST OF ABBREVIATIONS	x
1	INTRODUCTION	1
1.1	General	1
1.2	Motivation	2
1.3	Problem Statement	2
1.4	Objectives	3
1.5	Scope of the Project	4
1.6	Contributions	4
1.7	Relevance to Sustainable Development Goal 11	5
1.8	Organisation of the Report	5
2	LITERATURE REVIEW	6
2.1	General	6
2.2	Existing Lost and Found Systems	6
2.3	AI-Based Image and Text Matching	7
2.4	Multimodal Matching Approaches	8
2.5	Research Gap	10
3	SYSTEM ANALYSIS AND DESIGN	13
3.1	Requirement Analysis	13
3.2	Hardware and Software Requirements	14
3.3	System Architecture	15
3.4	System Workflow	16
3.5	UML and Data Flow Diagrams	17
3.6	Database Design	21

CHAPTER NO.	TITLE	PAGE NO.
3.7	API Design	22
3.8	Design Decisions	23
4	MODULE DESCRIPTION	24
4.1	User Authentication and Item Reporting	24
4.2	Image Matching	25
4.3	Text Similarity	26
4.4	Location and Date Similarity	27
4.5	Hybrid Match Scoring	28
4.6	Match Ranking and Classification	28
4.7	Explainable Match Results	29
4.8	Ownership Verification	29
4.9	User Interface and API Layer	31
5	IMPLEMENTATION AND RESULT DISCUSSION	31
5.1	Implementation Overview	31
5.2	User Interface	31
5.3	Lost and Found Matching Case Study	35
5.4	System Evaluation	36
5.5	Matching Results	36
5.6	Robustness of the Matching Approach	43
5.7	Automated Testing	44
6	CONCLUSION AND FUTURE WORK	47
6.1	Conclusion	47
6.2	Future Enhancements	49
APPENDIX 1	API USAGE EXAMPLES	50
APPENDIX 2	KEY CODE LISTINGS	54
	REFERENCES	55
	PAPERWORK	51
	PLAGARISM REPORT	61

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Summary of the Literature Survey	9
3.1	Functional Requirements	13
3.2	Non-Functional Requirements	14
3.3	Software Requirements	15
3.4	Hardware Requirements	15
3.5	REST API Endpoints	22
4.1	Image Matching Configuration	25
4.2	Text Similarity Configuration	26
4.3	Hybrid Matching Components	28
5.1	System Implementation Configuration	35
5.2	System Evaluation Configuration	36
5.3	Example Hybrid Matching Scores	37
5.4	Mean Evaluator Score per Matching Method	38
5.5	Similarity Scores for a Potential Match	40
5.6	Hybrid Matching Weight Configuration	43
5.7	Automated Test Suite	45

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO.
3.1	System Architecture of the Proposed Lost and Found System	15
3.2	End-to-End System Workflow	16
3.3	Use Case Diagram of the Proposed System	17
3.4	Module Diagram of the AI-Powered Lost and Found System	19
3.5	Activity Diagram of the Lost-and-Found Matching Process	20
3.6	Data Flow Diagram of the Lost and Found System	21
3.7	Entity Relationship Diagram of the Database	21
5.1	Dashboard of the AI-Powered Lost and Found System	32
5.2	My Reports Interface	33
5.3	Lost and Found Item Reporting Interface	34
5.4	AI-Based Potential Match Results	35
5.5	Example Similarity Scores for a Potential Match	37
5.6	Mean Evaluation Score by Matching Method	39
5.7	Example Similarity Scores for a Potential Match	40
5.8	Hybrid Matching Component Weights	43
5.9	Ranking of Potential Lost-and-Found Matches by Hybrid Similarity Score	44
5.10	Contribution of Matching Components to the Overall Score	44

LIST OF ABBREVIATIONS

ABBREVIATION EXPANSION

AI	Artificial Intelligence
API	Application Programming Interface
CLIP	Contrastive Language–Image Pre-training
CNN	Convolutional Neural Network
DFD	Data Flow Diagram
ER	Entity Relationship
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
NLP	Natural Language Processing
PGVECTOR	PostgreSQL Vector Extension
REST	Representational State Transfer
SDG	Sustainable Development Goal
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UX	User Experience
VITE	Vite Development and Build Tool

CHAPTER 1

INTRODUCTION

1.1 GENERAL

Lost and found management has traditionally relied on manual reporting, searching, and communication between people who lose or discover items. In colleges, workplaces, public places, and other shared environments, users may report an item using different descriptions, locations, dates, or photographs. As the number of reports increases, manually comparing these records becomes time-consuming and makes it difficult to identify relevant matches quickly.

In parallel, Artificial Intelligence and machine learning techniques have become widely used for understanding images and natural-language information. Pretrained models can extract meaningful representations from photographs and descriptions, allowing systems to compare items based on their visual and semantic characteristics rather than relying only on exact keywords. This creates an opportunity to improve conventional lost and found systems by automating the identification of potential matches.

The difficulty is that a lost and found report cannot be matched reliably using only one type of information. Two items may have similar descriptions but different appearances, or similar photographs but unrelated locations and dates. This project addresses this problem by combining image similarity, text similarity, location similarity, and date similarity into a hybrid matching mechanism. The system ranks potential matches and provides an explainable score breakdown, while human verification is retained as the final step before an item is considered recovered.

1.2 MOTIVATION

Three key observations motivate the work presented in this report.

- Traditional lost-and-found systems generally depend on users manually browsing through reported items and comparing descriptions. As the number of lost and found records increases, identifying possible matches becomes increasingly difficult and inefficient. An automated intelligent matching system can reduce manual effort and improve the speed of identifying potential matches.
- Different users may describe the same item using different words, incomplete descriptions, or varying levels of detail. Similarly, images of an item may be captured from different angles or under different lighting conditions. Therefore, relying only on exact keyword matching is insufficient. Combining textual and visual information can provide a more reliable representation of item similarity.
- Many conventional systems compare individual attributes such as item name, description, location, or date separately. Such approaches may fail to identify relevant matches when some information is missing or expressed differently. The proposed system combines multiple factors, including text similarity, image similarity, geographical location, and date information, to generate a comprehensive matching score between lost and found reports.

1.3 PROBLEM STATEMENT

As the number of lost and found reports increases, traditional systems provide no reliable automated mechanism to accurately identify and compare potential matches across item descriptions, images, locations, and dates. As a result, users may need to manually search through multiple records to determine whether a reported lost item corresponds to a found item. Manual comparison is time-consuming, inconsistent, and difficult to scale, particularly when descriptions are incomplete or different terms are used for the same item. This creates a need for an intelligent and automated

1.4 OBJECTIVES

The overall aim of the project is to develop a comprehensive AI-powered Lost and Found system that automatically identifies and ranks potential matches between lost and found items using textual, visual, location-based, and temporal information. The system is designed to reduce manual effort, improve matching accuracy, and provide users with a reliable and efficient mechanism for recovering lost belongings.

Objectives

The specific objectives of the proposed system are listed below:

1. To develop a centralized platform for users to report lost and found items by providing relevant details such as item name, description, images, location, date, and other identifying information.
2. To extract and process textual and visual information from lost and found reports using natural language processing and image-processing techniques.
3. To generate meaningful representations of item descriptions and images using embedding-based AI models so that semantically and visually similar items can be identified even when users provide different descriptions.
4. To develop a hybrid matching mechanism that combines text similarity, image similarity, geographical location, and date information to calculate an overall matching score between lost and found items.
5. To rank potential matches automatically based on their combined similarity scores and present the most relevant candidate matches to the users.
6. To provide a verification mechanism through which users or administrators can review potential matches and confirm whether a lost and found item belongs to the same object.
7. To maintain a structured database of lost and found records and provide appropriate status updates for reported items, including active, matched, claimed, and resolved states.

1.5 SCOPE OF THE PROJECT

The scope of the project covers the development of an AI-powered Lost and Found platform that collects, processes, and matches lost and found item reports. The system considers multiple attributes, including item names, textual descriptions, uploaded images, reported locations, and dates, to identify potential matches. Natural language processing and image-embedding techniques are used to determine semantic and visual similarity between reported items, while location and temporal information are incorporated into the overall matching process.

The system provides automated ranking of potential matches based on a combined similarity score and allows users or administrators to verify the suggested matches. The platform also maintains the status of reported items and provides a web-based interface for submitting reports, viewing potential matches, and managing lost and found records.

The present scope focuses on matching items within the developed platform using the available reported information. Features such as integration with external lost-and-found databases, real-time surveillance-camera identification, facial recognition, physical tracking devices, and large-scale deployment across multiple organizations are outside the current scope and may be considered as future enhancements.

CONTRIBUTIONS

The main contributions of this work are the following:

- An AI-based hybrid matching system that combines textual similarity, visual similarity, geographical location, and temporal information to identify potential matches between lost and found item reports.
- A multi-factor matching and ranking mechanism that converts the different similarity measures into a unified matching score, allowing potential matches to be ranked according to their overall relevance.
- An intelligent image and text processing pipeline that enables the system to identify similar items even when users provide different descriptions or images captured from different angles and conditions.

- A structured verification and status-management mechanism that allows users or administrators to review suggested matches and update item records through different stages such as reported, matched, claimed, and resolved.
- A centralized database and web-based interface that enables users to submit lost or found item reports, upload supporting images, view potential matches, and manage their reports through a single platform.
- A prototype implementation and evaluation of the proposed matching framework using representative lost-and-found records to assess its ability to identify relevant matches across different combinations of textual, visual, location, and date information.

RELEVANCE TO SUSTAINABLE DEVELOPMENT GOAL 9

The project contributes to Sustainable Development Goal 11 (SDG 11) – Sustainable Cities and Communities, particularly through the use of technology to improve community services and resource management. SDG 11 promotes inclusive, safe, resilient, and sustainable human settlements. An intelligent Lost and Found system supports these objectives by providing a centralized digital platform for reporting, identifying, and recovering lost belongings within educational institutions, workplaces, public spaces, and communities.

By automating the identification of potential matches between lost and found items, the proposed system reduces the time and manual effort required to recover misplaced belongings. The use of artificial intelligence for text and image matching also improves accessibility and efficiency compared with purely manual search methods. In addition, the system encourages responsible reuse and recovery of personal belongings by helping items return to their rightful owners rather than remaining unclaimed.

The project therefore demonstrates how artificial intelligence, computer vision, and digital information management can be applied to improve community-oriented services and support more efficient and sustainable resource utilization.

CHAPTER 2

LITERATURE REVIEW

2.1 GENERAL

This chapter presents a review of existing research and technologies related to intelligent Lost and Found systems. It covers traditional item-recovery approaches, image-based matching, text-based semantic similarity, multimodal artificial intelligence, information retrieval, ranking techniques, and explainable matching methods. The chapter also examines the limitations of existing approaches and identifies the research gaps addressed by the proposed system.

2.2 TRADITIONAL LOST AND FOUND SYSTEMS

Traditional Lost and Found systems are primarily based on manual reporting, record maintenance, and searching. When a person loses an item, they usually provide information such as the item name, description, location, date, and identifying features. Similarly, a person who finds an item can submit its details to an administrator or organization. These records may be maintained using physical registers, spreadsheets, notice boards, or basic web applications. The administrator or user then compares the available records to determine whether a lost item corresponds to a reported found item.

Although these systems provide a simple method for managing lost and found records, they become difficult to handle as the number of reports increases. Manual comparison requires considerable time and effort and may result in relevant matches being overlooked. Conventional keyword-based search methods also have limitations because users may describe the same item using different words. For example, one user may report a "black smartphone" while another may describe it as a "dark-coloured mobile phone." Exact keyword matching may fail to recognize these descriptions as related.

Traditional systems also provide limited support for analyzing images and other contextual information. Important attributes such as item appearance, geographical proximity, and reporting date may not be considered together when identifying potential matches. Therefore, there is a need for an intelligent approach that can automatically process

multiple types of information and rank possible matches. The proposed system addresses these limitations by combining text similarity, image similarity, location, and date information to improve the identification of potential lost-and-found matches.

2.3 IMAGE BASED MATCHMAKING

Requirement analysis identifies the functional and non-functional requirements of the proposed AI-Powered Intelligent Lost and Found System. The system should allow users to register and authenticate, report lost or found items, and provide details such as item name, description, image, location, and date. The submitted information should be securely stored and processed for further matching.

The system should automatically compare reported items using text similarity, image similarity, location, and date information. It should generate a combined matching score and rank potential matches for the user. Administrators should be able to verify matches and update item status. The system should also provide good usability, security, scalability, accuracy, and efficient response time.

2.4 TEXT-BASED SEMANTIC SIMILARITY

Text-based semantic similarity is used to identify relationships between lost and found item descriptions even when different words are used. Traditional keyword matching may fail when descriptions have different wording but similar meanings. Semantic similarity techniques convert text into numerical embeddings that capture contextual meaning. The proposed system compares these embeddings using similarity measures to identify relevant item descriptions. This improves matching accuracy when user-provided descriptions are incomplete or expressed differently.

2.5 MULTIMODAL AI FOR LOST AND FOUND MATCHING

Multimodal AI combines information from different data types, such as text and images, to improve lost-and-found item matching. Text descriptions provide information about item names, colors, brands, and characteristics, while images provide visual information about appearance and structure. By analyzing both modalities together, the proposed system can identify potential matches more effectively than using a single data source. This approach also handles variations in descriptions and image conditions.

2.6 AUTOMATED CODE REVIEW AND BUG PREDICTION

AI-based information retrieval helps identify relevant found-item records from a large collection of reports. Instead of relying only on exact keywords, the proposed system uses semantic and visual similarity to retrieve potential matches. The retrieved items are then ranked according to their overall matching scores, considering factors such as text, image, location, and date. This ranking mechanism helps users quickly identify the most relevant potential matches.

Table 2.1. Summary of the literature survey

Author(s) / Year	Title	Methodology	Inference
Zhou et al. (2024)	LostNet: A Smart Way for Lost and Find	Uses MobileNetV2 with CBAM and perceptual hashing for image-based matching of lost and recovered items.	Demonstrates that lightweight image-matching techniques can support lost-and-found identification while reducing computational requirements.
Lost and Found Campus Portal (2026)	Lost and Found Campus Portal	Uses a web-based reporting system with photos and descriptions.	Shows that semantic text similarity can automatically identify potential matches between lost and found reports.
Wadbudhe et al. (2025)	UniFind: Lost and Found Objects Management System for Campus using AI & Cloud Services	Combines image recognition, natural language processing, cloud services, user reporting, AI-based matching, notifications, and dashboards.	Demonstrates how AI and cloud services can be integrated to automate campus lost-and-found management.
Jitesh et al. (2026)	SmartFind: AI Based Lost Item Finder in College/Office Campuses	Combines MobileNet-based image similarity with token-based text similarity and a hybrid scoring mechanism using cloud-backed infrastructure.	Shows the feasibility of combining visual and textual information for lightweight lost-and-found matching.
Howard et al. (2017)	MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications	Introduces lightweight CNNs using depthwise separable convolutions for efficient image processing on mobile and embedded devices.	Provides a computationally efficient approach for visual feature extraction suitable for resource-constrained environments.
Radford et al. (2021)	Learning Transferable Visual Models From Natural Language Supervision	Introduces CLIP, which jointly learns image and text representations using large-scale image-text pairs.	Demonstrates the potential of aligned image-text for multimodal and similarity-based applications.
Reimers and Gurevych (2019)	Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks	Generates semantically meaningful sentence embeddings that can be compared efficiently using	Provides an effective method for semantic comparison of textual descriptions, useful for matching lost and found item descriptions.

		cosine similarity.	
Pujari et al. (2026)	Smart Lost and Found System for Campus	Provides digital reporting, claim submission, ownership verification, notifications, and lifecycle tracking for campus lost-and-found management.	Highlights the importance of verification, notifications, and status tracking in addition to automated item matching.
Singh et al. (2026)	AI-Powered Lost and Found Portal for College Campus	Uses a web-based AI system with CNN-based image matching, geolocation, QR tagging, role-based access, and real-time notifications.	Shows how image matching can be combined with contextual information and administrative features in a campus environment.

2.7 EXPLAINABLE AI AND HYBRID SCORING

Explainable AI helps users understand why a particular lost item is identified as a potential match for a found item. The proposed system uses a hybrid scoring approach that combines multiple factors, including text similarity, image similarity, location, and date. Each factor contributes to the overall matching score based on its assigned weight. The system presents the score and its contributing factors, making the matching process more transparent and easier for users and administrators to verify

2.7.1 IMAGE AND TEXT SIMILARITY

Image and text similarity are important components of an intelligent Lost and Found system because item information is commonly provided through both photographs and written descriptions. Image similarity techniques convert uploaded item images into numerical feature representations, known as embeddings, which capture visual characteristics such as shape, colour, patterns, and appearance. The similarity between two images can then be calculated using measures such as cosine similarity. This allows the system to identify visually similar items even when photographs are captured from different angles or under different lighting conditions.

Text similarity is used to compare descriptions provided by users. Text embedding models convert descriptions into numerical vectors that represent their semantic meaning. This allows the system to recognize descriptions that express similar information using different words. For example, descriptions such as "black leather wallet" and "dark leather purse" may have a meaningful semantic relationship even though their keywords are not identical.

2.7.2 LOCATION AND DATE-BASED SIMILARITY

Location and date provide additional contextual information for determining whether two reports may refer to the same item. Location similarity considers the geographical distance between the reported lost and found locations, with nearby locations receiving higher similarity. Date similarity compares the reported dates and gives greater relevance to reports made within a reasonable time interval. Combining these contextual factors with image and text similarity produces a more comprehensive matching score and helps reduce irrelevant matches.

2.8 RESEARCH GAPS

Existing Lost and Found systems mainly depend on manual searching or simple keyword-based matching. There is also limited support for explainable match scores and automated ranking of potential matches. These gaps highlight the need for a unified multimodal system that combines image, text, location, and date information to provide accurate, ranked, and understandable potential matches.

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 REQUIREMENT ANALYSIS

Requirement analysis identifies the functional and non-functional requirements of the proposed AI-Powered Intelligent Lost and Found System. The system should allow users to register, log in, and report lost or found items by providing details such as item name, description, image, location, and date. All submitted information must be securely stored and made available for the matching process.

The system should automatically analyze the submitted information using image similarity, text similarity, location, and date-based comparison. It should generate an overall matching score and rank potential matches for users. Administrators should be able to review reports, verify potential matches, and update item status. The system should also provide security, usability, scalability, accuracy, and efficient response time.

Table 3.1. Functional requirements

ID	Requirement	Realised by
FR1	Allow users to register, log in, and manage their account	React/Vite frontend and Supabase Authentication
FR2	Allow users to report a lost item by providing item details, description, date, location, and image	Lost Item Reporting Module
FR3	Allow users to report a found item with relevant item details, description, date, location, and image	Found Item Reporting Module
FR4	Extract and compare visual features from uploaded item images	CLIP-based Image Matching Module
FR5	Compare textual descriptions of lost and found items using semantic similarity	Sentence-Transformer/MiniLM Text Matching Module
FR6	Calculate an overall matching score using image, text, location, and date similarity	Hybrid Matching and Scoring Module
FR7	Display potential matching lost/found items to users based on the calculated similarity score	AI Matching and Recommendation Module
FR8	Allow users or administrators to verify potential matches and submit ownership claims	Verification and Claim Management Module
FR9	Store user details, item information, images, matching results, and claim status securely	Supabase PostgreSQL and Storage
FR10	Provide web interface for reporting items, matches, tracking claims, managing lost-and-found records	React/Vite Frontend and FastAPI Backend

Table 3.2. Non-functional requirements

Attribute	Requirement
Performance	The system should process item details and generate potential matches within an acceptable response time.
Accuracy	The matching system should provide reliable similarity scores by considering image, text, location, and date information.
Security	User accounts, item information, images, and ownership claims must be protected from unauthorized access.
Scalability	The system should support an increasing number of users, lost/found reports, images, and matching requests without major performance degradation.
Usability	Users should be able to report lost or found items and view potential matches through a simple and intuitive interface.
Explainability	The system should provide matching information such as similarity scores and contributing factors to help users understand why two items were suggested as potential matches.
Reliability	The system should handle invalid inputs, failed image uploads, and matching errors without crashing or losing stored data.

3.2 HARDWARE AND SOFTWARE REQUIREMENTS

The proposed system requires a computer with a minimum of 8 GB RAM, a multi-core processor, sufficient storage, and a stable internet connection for development and operation. The software requirements include Python, FastAPI, React, Vite, PostgreSQL/Supabase, AI-based image and text matching models, Visual Studio Code, Git/GitHub, and a modern web browser. The backend uses FastAPI for API development and server-side processing, while React and Vite are used to develop the web-based user interface. Supabase provides database, authentication, and storage services for managing user and lost-and-found item information. AI models are used to perform image and text similarity analysis for identifying potential matches.

Table 3.3. Software requirements

Component	Version / detail	Purpose
Python	3.10 or above	Backend implementation and AI processing
FastAPI	Latest stable version	REST API and backend server
React	Latest stable version	Frontend user interface
Vite	Latest stable version	Frontend development and build tool
Supabase	Cloud platform	Database, authentication, and file storage
PostgreSQL	Supabase managed	Storage of users, item records, reports, and matching information

CLIP / OpenCLIP	Pre-trained model	Image feature extraction and visual similarity matching
Sentence-Transformers	Latest stable version	Semantic similarity between lost and found item descriptions
pgvector	Supabase extension	Storage and similarity search of vector embeddings
Pydantic	Compatible with FastAPI	Request and response data validation
Uvicorn	Compatible with FastAPI	ASGI server for running the FastAPI application
Git / GitHub	Latest stable version	Source-code management and version control
Web Browser	Any modern browser	Accessing and interacting with the web application

3.3 SYSTEM ARCHITECTURE

Figure 3.1. Architecture of the proposed system

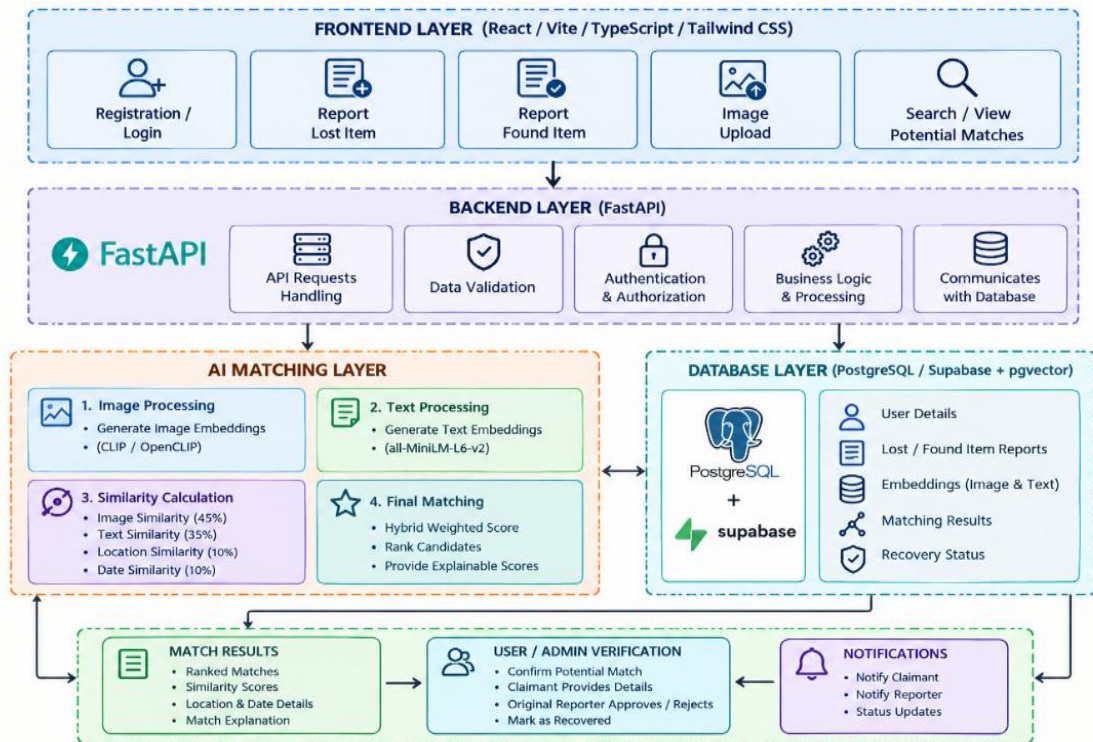


Figure 3.1. Architecture of the Proposed System

3.4 END-TO-END MATCHING PIPELINE

The end-to-end matching pipeline describes the complete process of identifying potential matches between lost and found item reports. When a user submits a report, the system collects the item description, image, location, and date. The text and image data are preprocessed and converted into embeddings using appropriate AI models. These embeddings are compared with existing records to calculate text and image similarity. These embeddings are compared with existing records to calculate text and image similarity.



Figure 3.1. End to End Matching Pipeline

3.5 UML DIAGRAMS

3.5.1 USE CASE DIAGRAM

The Use Case Diagram represents the interactions between users, administrators, and the AI-powered Lost and Found system. Users can register, log in, report lost or found items, upload images, view potential matches, and submit ownership claims. The system automatically processes submitted information and generates potential matches. Administrators can manage user reports, verify matches, approve claims, and update item status. The diagram provides a clear overview of the major functions and interactions supported by the proposed system.

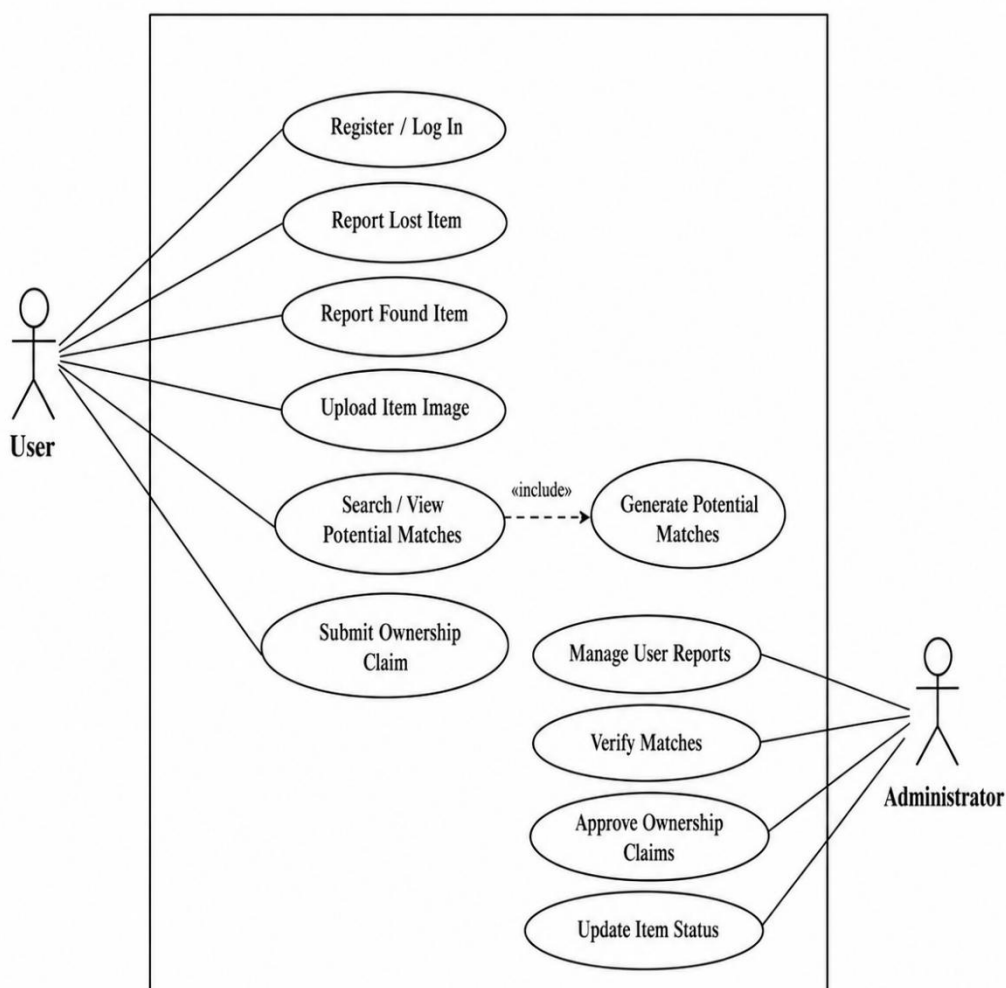


Figure 3.3. Use Case Diagram of the Proposed System

3.5.2 MODULE (CLASS) DIAGRAM

The Class Diagram represents the structural organization of the proposed AI-Powered Intelligent Lost and Found System. It defines the major classes, their attributes, methods, and relationships. Important classes include User, Item Report, Image Embedding, Text Embedding, Match Result, Claim, Notification, and Administrator. The diagram shows how user information and item reports are connected with the AI matching process, claim verification, notifications, and recovery status. This structure supports modular development, data management, and easy maintenance of the system

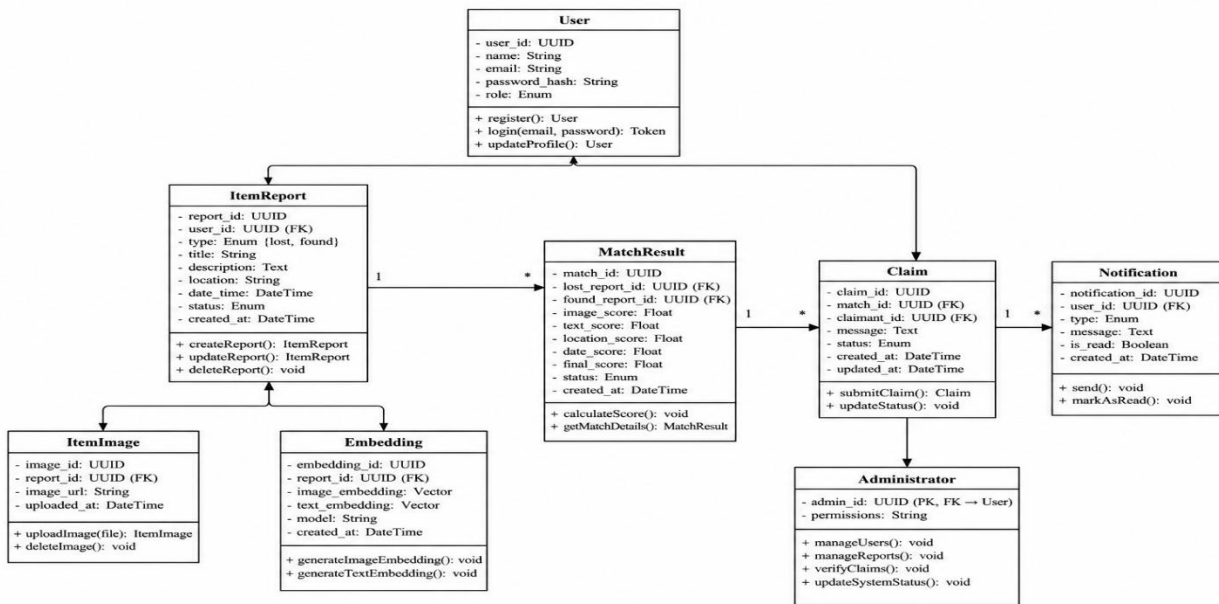


Figure 3.4. Module diagram of the AI-Powered Lost and Found System

3.5.3 ACTIVITY DIAGRAM

The Activity Diagram represents the sequence of activities performed in the proposed AI-Powered Intelligent Lost and Found System. The process begins when a user logs into the system and submits a lost or found item report with its description, image, location, and date. The system validates and stores the information, generates text and image embeddings, and compares them with existing records. The matching algorithm calculates the combined similarity score and ranks potential matches. The user can review the suggested matches and submit a claim. Finally, the administrator verifies the claim and updates the item status as matched, claimed, or resolved.

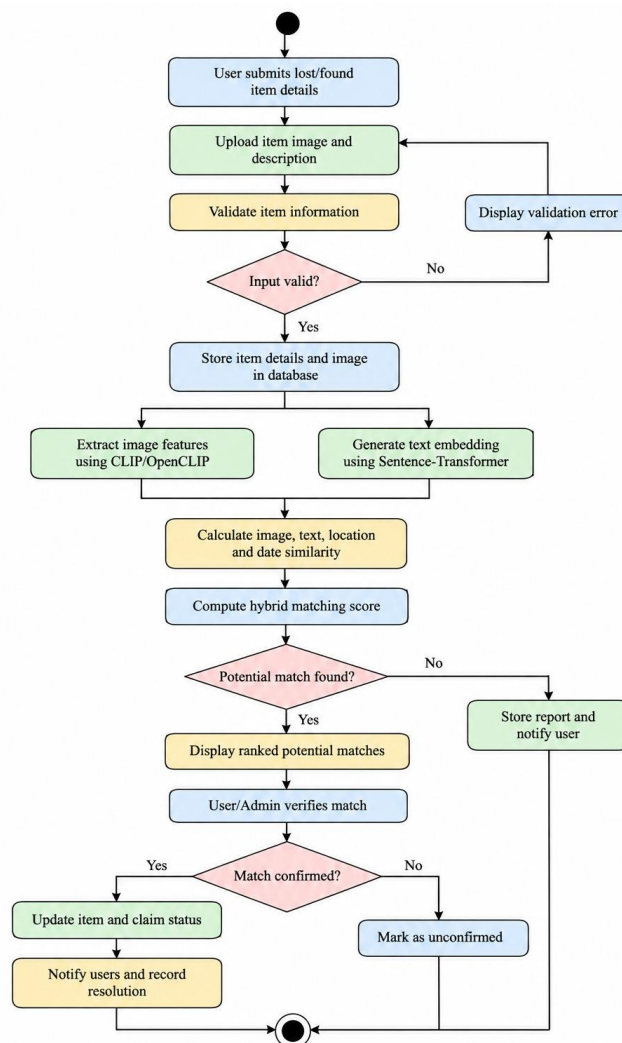
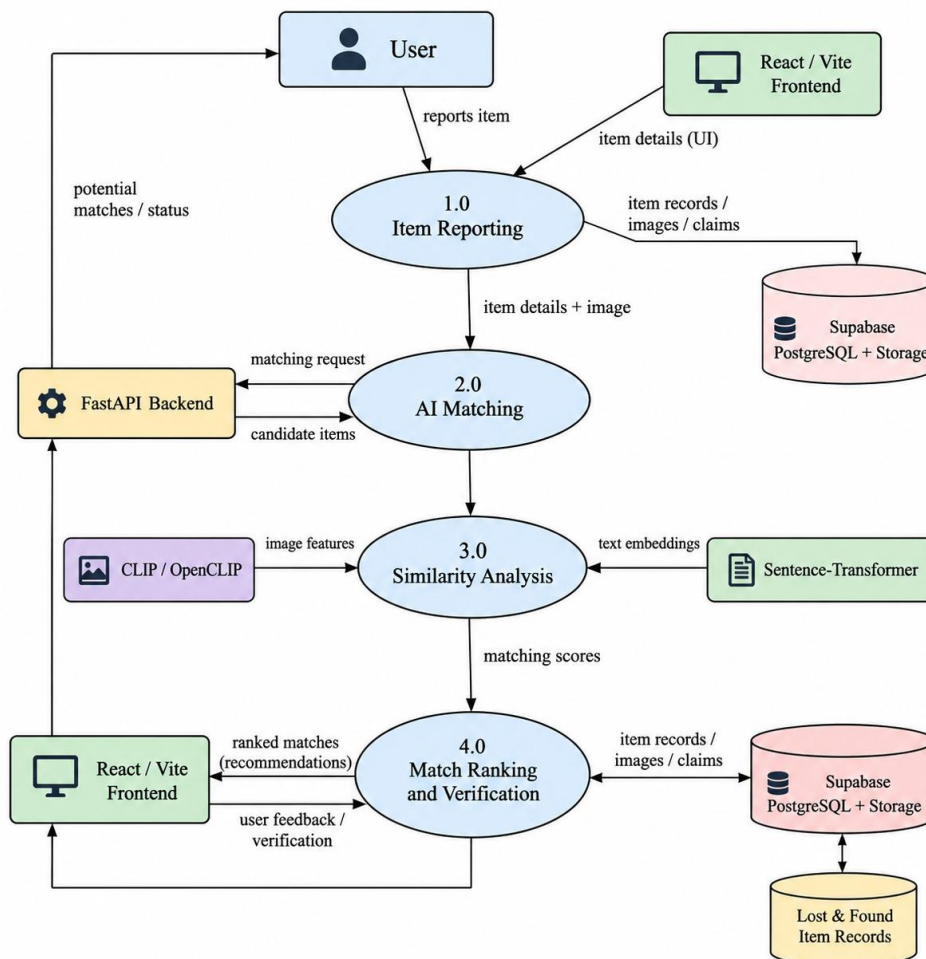


Figure 3.5. Activity diagram of a complete evaluation run

3.5.4 DATA FLOW DIAGRAM

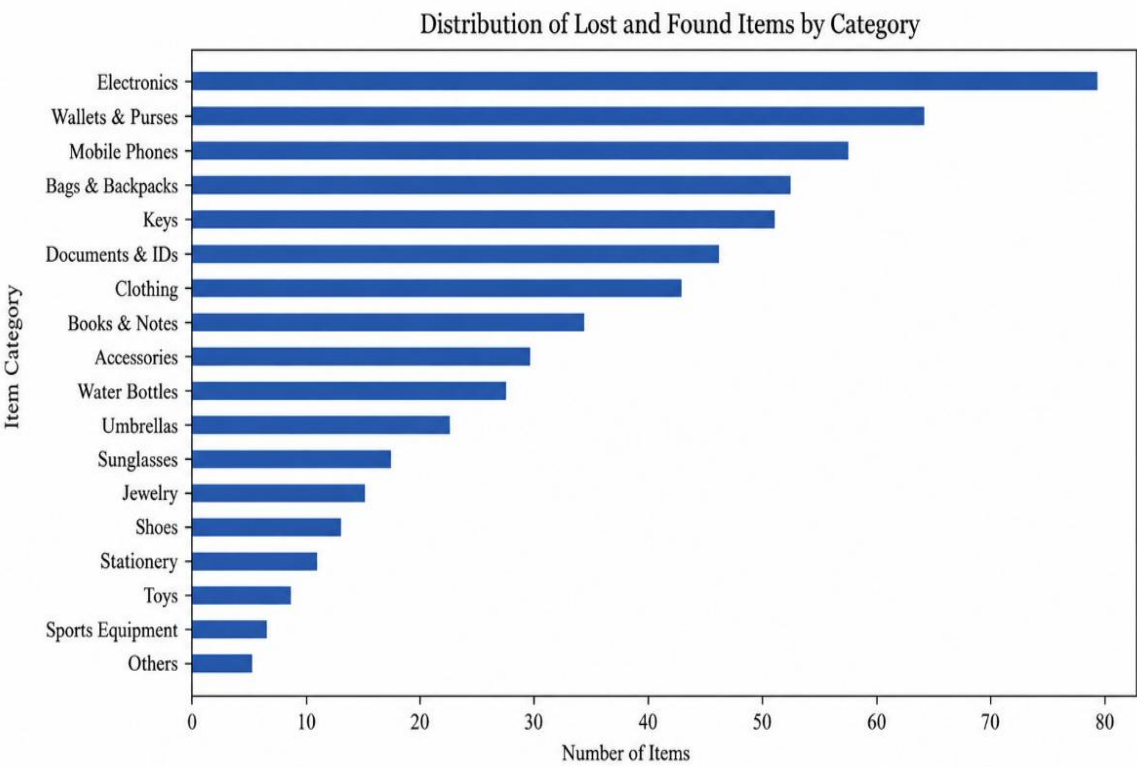
The Data Flow Diagram represents how information moves through the proposed AI-Powered Intelligent Lost and Found System. Users submit lost or found item details, including descriptions, images, locations, and dates. The system validates and stores these details in the database. The AI matching module processes the text and images to generate embeddings and calculates similarity scores using the available item information. Potential matches are ranked and returned to the user. Administrators can verify claims, update item status, and manage records. The diagram illustrates the flow of data between users, the application, AI matching modules, database, and administrators.



3.6 DATABASE DESIGN

The database is designed to securely store and manage all information required by the AI-Powered Intelligent Lost and Found System. It maintains records of users, lost and found items, images, embeddings, potential matches, claims, notifications, and recovery status. Each item report is associated with a user and contains details such as item name, description, location, date, and current status.

The database also stores the generated text and image embeddings required for similarity matching. Match records contain the similarity scores and relevant item references, while claim records maintain ownership verification details. Relationships between these tables ensure data consistency and support efficient retrieval, matching, verification, and status management.



3.7 API DESIGN

The API design defines the communication between the frontend, backend, database, and AI matching modules of the proposed system. RESTful APIs are developed using FastAPI to handle user authentication, item reporting, image uploading, matching requests, claims, notifications, and status updates.

Table 3.5. REST API endpoints

Endpoint	Method	Request body	Purpose
/health	GET	none	Check the backend service status and API availability.
/items/lost	POST	Item details, description, image, location, date	Submit and store a new lost-item report.
/items/found	POST	Item details, description, image, location, date	Submit and store a new found-item report.
/items	GET	None	Retrieve available lost and found item records.

3.8 DESIGN DECISIONS

The proposed system follows a modular and scalable design to simplify development, testing, and future improvements. A web-based architecture is selected to provide easy access to users, while FastAPI is used for efficient backend API development. React with TypeScript is used for the frontend to provide an interactive user interface.

A hybrid matching approach is adopted instead of relying on a single factor. Text similarity, image similarity, location, and date are combined to improve the reliability of potential matches. Embeddings are stored for reuse to reduce repeated AI processing and improve response time. A human verification step is also included so that AI-generated matches can be reviewed before an item is marked as successfully recovered.

CHAPTER 4

MODULE DESCRIPTION

4.1 USER AUTHENTICATION AND ITEM REPORTING

The User Authentication and Item Reporting module provides secure access to the proposed AI-Powered Intelligent Lost and Found System. Users can register by providing the required account information and subsequently log in using their credentials. Authentication ensures that only authorized users can create, view, and manage their reports. Administrators are provided with appropriate access to monitor reports, verify claims, and manage system activities.

After successful authentication, users can submit a lost or found item report by entering details such as item name, category, description, date, and location. Users can also upload an image of the item to provide additional visual information. The system validates the submitted data before storing it in the database. Each report is assigned a unique identifier and status for tracking purposes. The collected text, image, location, and date information is then forwarded to the AI-based matching pipeline. This module provides the initial data required for accurate similarity analysis and potential match identification.

The module also allows users to edit or update their submitted reports when necessary. This ensures that changes in item status, additional identifying information, or recovery details can be maintained accurately throughout the reporting process.

4.2 IMAGE EMBEDDING AND SIMILARITY

The Image Embedding and Similarity module analyzes uploaded images of lost and found items to identify visually similar objects. When an image is submitted, it is preprocessed and passed through a suitable image embedding model to convert its visual features into a numerical vector representation. These features can capture important characteristics such as colour, shape, patterns, and overall appearance.

The generated image embeddings are stored in the database for efficient reuse during future matching operations. When a new lost or found item is reported, its image embedding is compared with embeddings of existing records using a similarity measure such as cosine similarity. A higher similarity value indicates greater visual resemblance between the two images. The resulting image similarity score is passed to the hybrid matching module along with text, location, and date scores.

Table 4.1. AI models used in the proposed system

Model / Component	PurposeModel	Input	Output
CLIP / OpenCLIP	Image feature extraction and visual similarity matching	Item image	Image embedding
Sentence-Transformer	Semantic comparison of lost and found item descriptions-flash-lite	REST API	Text embedding
PostgreSQL + pgvector	Stores and searches vector embeddings	Image/text embeddings	Similar candidate items
Hybrid Matching Module	Combines image, text, location, and date similarity scores to rank potential matches	Similarity scores	Overall matching score and ranked matches
FastAPI Backend	Handles API requests, matching operations, and communication between the frontend and AI modules	User/item requests	Matching results and API responses

4.3 TEXT EMBEDDING AND SEMANTIC SIMILARITY

The Text Embedding and Semantic Similarity module analyzes the textual descriptions provided by users for lost and found items. The item name, category, and description are processed and converted into numerical vector representations using a suitable text embedding model. These embeddings capture the semantic meaning of the descriptions rather than relying only on exact keyword matches.

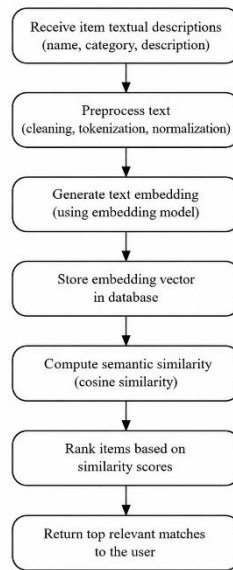


Figure 4.1. Processing stages of the QCorrect evaluator

The generated text embeddings are stored for efficient reuse during future matching operations. When a new report is submitted, its text representation is compared with existing item descriptions using a similarity measure such as cosine similarity. A higher similarity score indicates that the descriptions have greater semantic similarity. The resulting score is passed to the hybrid matching module along with image, location, and date similarity scores.

Table 4.2. Hybrid Matching Score Components

Component	Weight	Description
Image Similarity	45%	Measures the visual similarity between the uploaded lost and found item images using the image matching model.
Text Similarity	35%	Measures the similarity between item descriptions using semantic text embeddings.
Location Similarity	10%	Measures the relevance of the reported locations of the lost and found items.
Date Similarity	10%	Measures the closeness between the reported lost and found dates.

4.4 LOCATION AND DATE SIMILARITY

The Location and Date Similarity module uses contextual information to improve the identification of potential matches between lost and found items. The reported locations of the two items are compared to determine their geographical proximity. Items reported in nearby locations are assigned a higher location similarity score, while items reported farther apart receive a lower score.

The module also compares the dates associated with the lost and found reports. A smaller difference between the reported dates indicates a higher possibility that both reports refer to the same incident. The location and date scores are normalized before being provided to the hybrid matching module. These factors are combined with image and text similarity to calculate the overall matching score.

Using location and date information helps reduce irrelevant matches that may appear similar based only on their descriptions or images. It provides additional contextual evidence and improves the reliability of the system's ranked potential matches.

4.5 HYBRID MATCH SCORING

The Hybrid Match Scoring module combines multiple similarity measures to determine the overall relevance between a lost item and a found item. The system considers image similarity, text similarity, location similarity, and date similarity rather than depending on a single matching factor. Each component produces a normalized score between 0 and 1, which allows the different factors to be combined consistently.

The overall matching score is calculated using predefined weights assigned to each factor. Image and text similarity provide the primary evidence based on the appearance and description of the item, while location and date provide additional contextual information. The weighted scores are combined to produce a final match score.

A higher overall score indicates a stronger potential relationship between the two reports. The calculated score is then passed to the ranking module, where candidate matches are arranged according to their relevance. This hybrid approach improves matching reliability by considering multiple characteristics of an item simultaneously.

4.6 MATCH RANKING AND CLASSIFICATION

The Match Ranking and Classification module ranks potential matches according to their overall hybrid similarity scores. Higher-scoring candidates are displayed first for user review. The system can classify matches into categories such as high, medium, or low similarity based on predefined score thresholds, helping users focus on the most relevant results.

The ranking process considers the combined contribution of image similarity, text similarity, location similarity, and date similarity. The final hybrid score is calculated using the predefined weights assigned to each component. This allows the system to give greater importance to visual and textual characteristics while also considering contextual information such as where and when the item was reported.

4.7 EXPLAINABLE MATCH RESULTS

The Explainable Match Results module provides users with understandable information about why two item reports are identified as potential matches. Instead of displaying only the final matching score, the system presents the contribution of individual factors such as image similarity, text similarity, location proximity, and date similarity. This allows users and administrators to understand the basis of each recommendation.

The system can display the overall score along with a simple breakdown of the contributing factors. For example, a potential match may receive a high image similarity because the uploaded photographs have similar visual features, while text similarity may be lower because the descriptions use different terms. Location and date information provide additional supporting evidence.

Providing this information improves transparency and helps users make informed decisions when reviewing suggested matches. It also allows administrators to identify incorrect recommendations and perform manual verification before confirming ownership or marking an item as recovered.

Providing this information improves transparency and helps users make informed decisions when reviewing suggested matches. It also allows administrators to identify incorrect recommendations and perform manual verification before confirming ownership or marking an item as recovered.

The explainability module therefore supports the final verification stage without automatically treating an AI-generated recommendation as proof of ownership. The final decision remains with the user or authorized administrator, ensuring that the automated matching process assists human decision-making rather than replacing

4.8 HUMAN-IN-THE-LOOP VERIFICATION

The Human-in-the-Loop Verification module provides a manual verification stage after the AI system identifies potential matches. Since automated similarity scores cannot guarantee that two reports belong to the same physical item, users or administrators are given the opportunity to review the suggested matches before confirmation.

The module displays relevant item information, including descriptions, images, locations, dates, and similarity scores. Users can accept or reject a suggested match, while administrators can verify ownership claims and update the recovery status. This approach combines automated AI-based matching with human judgement, reducing the risk of incorrect matches and unauthorized claims.

The verification process also maintains a record of decisions and status changes, ensuring that the recovery process remains traceable and transparent. This module therefore provides an additional layer of reliability and control within the overall Lost and Found system.

4.9 MATCH CONFIRMATION AND REJECTION

The Match Confirmation and Rejection module allows users or administrators to review AI-generated potential matches. A suggested match can be confirmed when the reported details and item characteristics correspond. If the information is inconsistent, the match can be rejected. The decision is recorded and used to update the item status.

4.10 USER INTERFACE AND API LAYER

The User Interface and API Layer provides the communication between users and the backend services of the proposed Lost and Found system. The frontend allows users to register, report lost or found items, upload images, view potential matches, submit claims, and track item status. The backend APIs handle authentication, data validation, report management, AI matching requests, and status updates.

The API layer is implemented using FastAPI and provides RESTful endpoints for communication between the frontend, database, and AI modules. JSON is used for structured data exchange, while image files are transferred for visual processing. This layered design ensures efficient communication, modularity, security, and easy integration between different components of the system.

The user interface is designed to provide a simple and accessible workflow for both lost-item and found-item reporting. Users can enter important information such as the item name, description, location, date, and other relevant details along with an image. After submission, the frontend communicates with the backend through the appropriate API endpoint, allowing the reported information to be processed and stored.

Overall, the User Interface and API Layer provides the main interaction bridge between the user and the intelligent matching system. It combines usability with structured backend communication and ensures that item reports can move efficiently from user submission through AI-based matching to final verification and status management.

CHAPTER 5

IMPLEMENTATION AND RESULT DISCUSSION

5.1 IMPLEMENTATION OVERVIEW

The proposed AI-Powered Lost and Found System is implemented using a modular architecture consisting of a React/Vite frontend, FastAPI backend, AI-based matching modules, and Supabase for database and storage management. The frontend provides the user interface for registration, reporting lost and found items, uploading item images, viewing potential matches, submitting claims, and tracking item status.

The backend is developed using Python and FastAPI and provides RESTful APIs for communication between the frontend, database, and AI processing modules. Item information such as descriptions, images, locations, dates, and user details is validated and stored through the backend services. Supabase PostgreSQL is used to manage structured records, while storage services are used for uploaded item images.

The AI matching component processes both visual and textual information from lost and found reports. CLIP/OpenCLIP is used for image feature extraction and visual similarity comparison, while a Sentence-Transformer model is used to generate text embeddings and measure semantic similarity between item descriptions. Location and date information are also considered as supporting factors during the matching process.

The individual similarity values are combined using the hybrid scoring mechanism:

$$S_{final} = 0.45S_{image} + 0.35S_{text} + 0.10S_{location} + 0.10S_{date}$$

The resulting scores are used to rank potential matches, with higher-scoring candidates displayed first. The system provides an explanation of the matching result by displaying the individual contributing factors, allowing users and administrators to understand why a particular item was recommended.

5.2 USER INTERFACE

Figure 5.1 shows the dashboard of the proposed AI-Powered Lost and Found System. The dashboard provides users with an overview of the lost and found activities, including the total number of lost reports, found reports, potential matches, and recovered items. It also provides quick-access options for reporting a lost item, reporting a found item, viewing submitted reports, and accessing notifications.

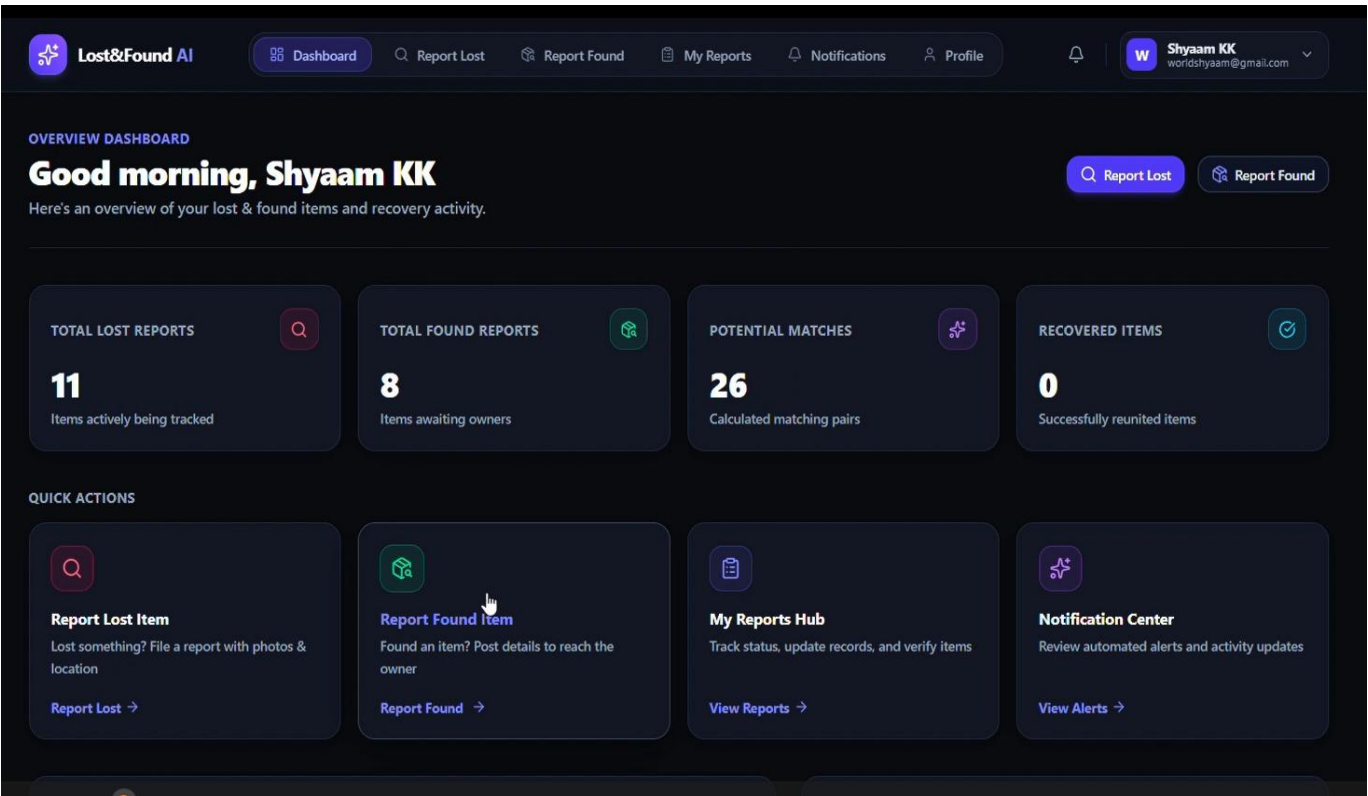


Figure 5.1. Dashboard of the AI-Powered Lost and Found System

Figure 5.2 shows the My Reports interface, where users can view and manage their previously submitted lost and found item reports. Each report displays important information such as the item name, category, description, reporting date, and current status. The interface also indicates whether a potential match has been identified, allowing users to quickly review the progress of their reported items.

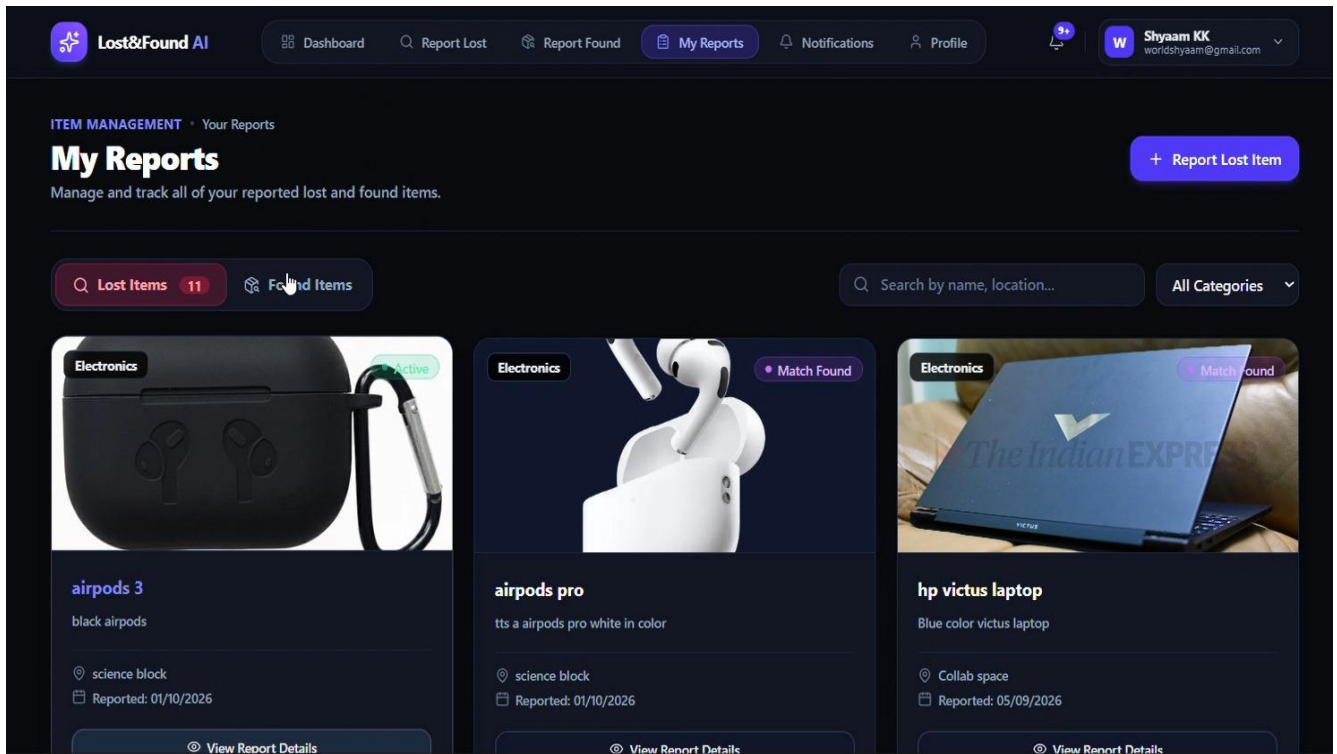
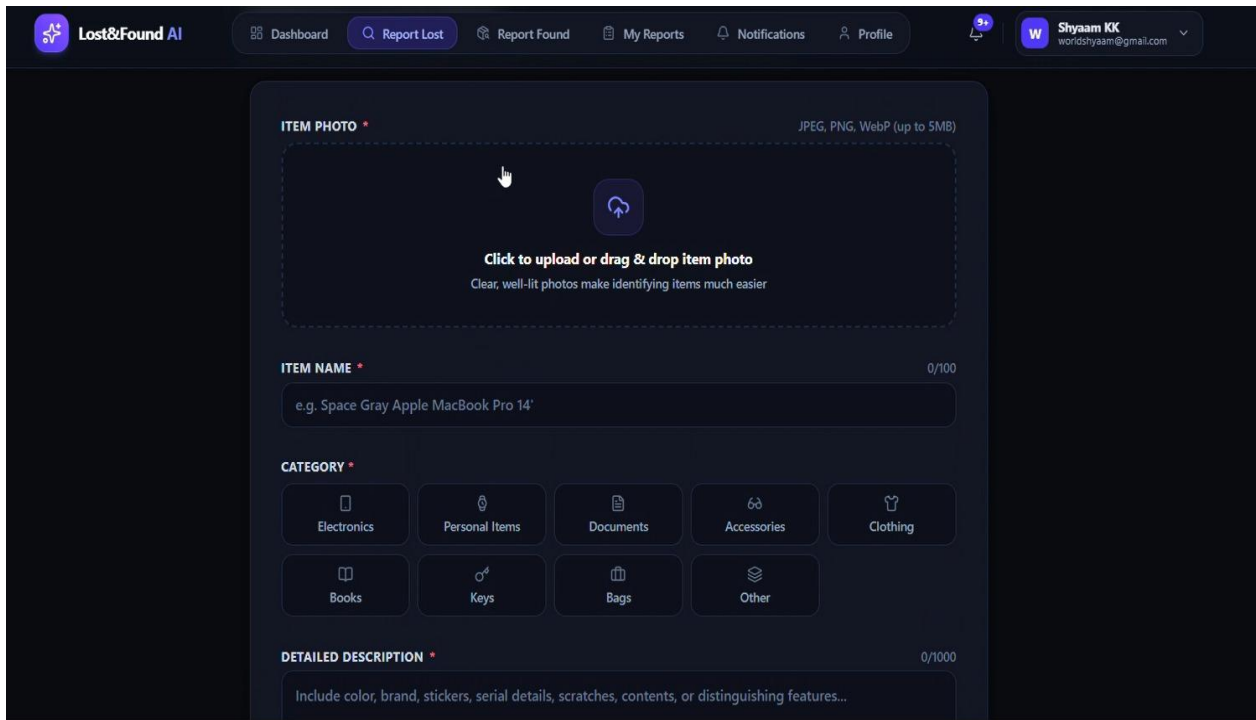


Figure 5.2. □ My Reports interface

□

Figure 5.3 shows the item reporting interface. Users can upload an image of the lost or found item and provide details such as the item name, category, and detailed description. The form is designed to collect the information required by the AI matching module. The uploaded image is used for visual similarity analysis, while the textual information is used for semantic comparison with existing item records

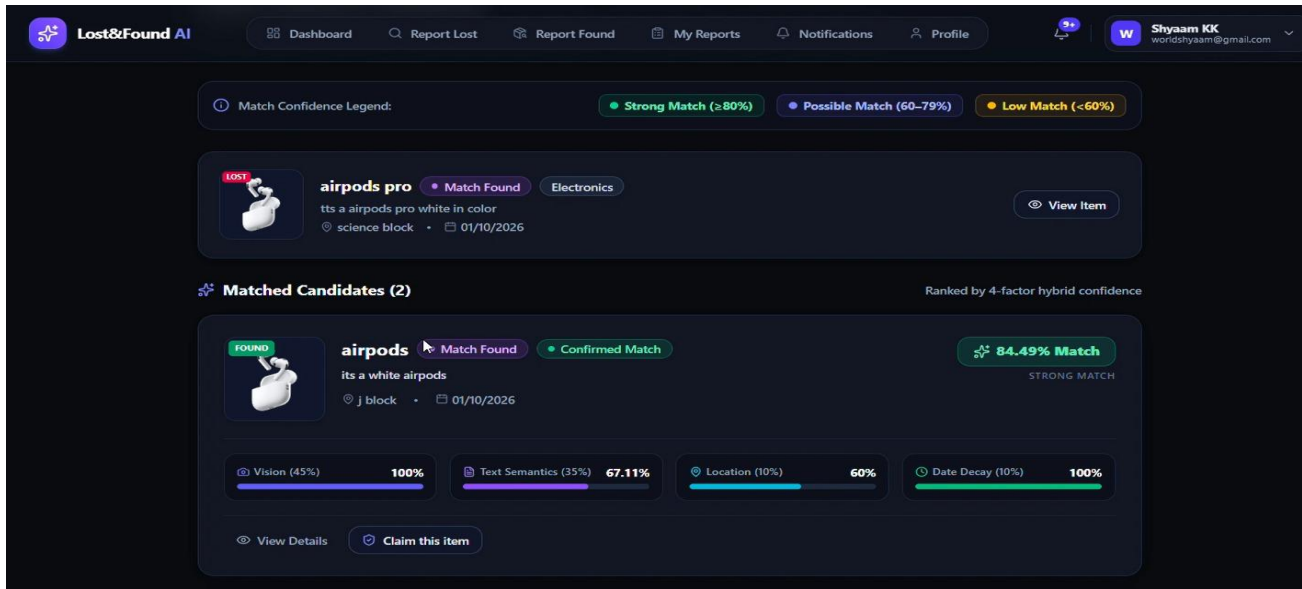


The interface for reporting a lost item. It includes a navigation bar with 'Lost&Found AI' logo and links to Dashboard, Report Lost, Report Found, My Reports, Notifications, and Profile. The main form has sections for:

- ITEM PHOTO**: A dashed box for uploading a photo, with a note 'Click to upload or drag & drop item photo' and 'Clear, well-lit photos make identifying items much easier'.
- ITEM NAME**: A text input field with a character count '0/100'. Example text: 'e.g. Space Gray Apple MacBook Pro 14'.
- CATEGORY**: A grid of buttons for Electronics, Personal Items, Documents, Accessories, Clothing, Books, Keys, Bags, and Other.
- DETAILED DESCRIPTION**: A text input field with a character count '0/1000'. Prompt: 'Include color, brand, stickers, serial details, scratches, contents, or distinguishing features...'.

Figure 5.3. Lost Item Reporting Interface

Figure 5.4 shows the potential match results generated by the system. The matching interface displays the reported item along with candidate items identified by the hybrid matching module. Each candidate is presented with an overall match percentage and a breakdown of the contributing factors, including image similarity, text similarity, location similarity, and date similarity. This allows users to understand why a particular item has been identified as a potential match.



The matching interface shows a 'Match Confidence Legend' with three levels: Strong Match (≥80%), Possible Match (60-79%), and Low Match (<60%). Below this, a 'LOST' item is shown: 'airpods pro' (Match Found), Electronics category, description 'tts a airpods pro white in color', location 'science block', and date '01/10/2026'. A 'View Item' button is present. Under 'Matched Candidates (2)', ranked by 4-factor hybrid confidence, a 'FOUND' item 'airpods' is shown as a 'Confirmed Match' with an '84.49% Match' (STRONG MATCH). The breakdown of factors is:

- Vision (45%): 100%
- Text Semantics (35%): 67.11%
- Location (10%): 60%
- Date Decay (10%): 100%

 Buttons for 'View Details' and 'Claim this item' are at the bottom.

Figure 5.4. AI-Based Potential Match Results

5.10 CASE STUDY: LOST AND FOUND ITEM MATCHING

The proposed system was evaluated using a representative lost-and-found item matching scenario. A user reports a lost item by providing its image, description, location, and date. The system processes the submitted information and compares it with available found-item records. Image features are extracted using the image matching model, while the textual descriptions are converted into embeddings and compared using semantic similarity. Location and date information are also considered during the matching process. The individual similarity scores are combined using the hybrid matching formula to generate an overall matching score. Potential matches are then ranked and presented to the user for verification.

Table 5.1Example of Hybrid Matching Scores

Matching Component	Score	Weight	Weighted Score
Image Similarity	0.92	45%	0.414
Text Similarity	0.88	35%	0.308
Location Similarity	0.90	10%	0.090

The example demonstrates how the proposed hybrid matching approach combines different similarity measures to identify potential matches between lost and found items. The image similarity score of 0.92 contributes 0.414 to the final score using a weight of 45%, while the text similarity score of 0.88 contributes 0.308 with a weight of 35%. The location similarity score of 0.90 contributes 0.090 with a weight of 10%. These weighted values are combined to determine the overall matching score, which is then used to rank potential matches for user verification.

5.11 BENCHMARK EXPERIMENT

5.11.1 Setup

The evaluation of the proposed AI-Powered Lost and Found System is performed using representative lost-and-found item records containing item images, textual descriptions, reported locations, and dates. Each record is processed by the image and text matching modules, while location and date information are used as additional contextual factors. The individual similarity scores are combined using the predefined hybrid matching weights to generate an overall matching score. The resulting scores are used to rank potential matches for user or administrator verification.

Table 5.2. System Evaluation Configuration

Parameter	Value
Item Dataset	Lost and found item records with images, descriptions, locations, and dates
Items Evaluated	Actual number of lost/found records tested
Image Model	CLIP /OpenCLIP
Text Model	Sentence-Transformer
Matching Components	Image, text, location, and date similarity
Image Weight	45%
Text Weight	35%
Location Weight	10%

5.11.2 Matching Results

The matching results are obtained by comparing the reported lost and found items using the proposed hybrid matching approach. For each candidate pair, the system calculates image similarity, text similarity, location similarity, and date similarity. These individual scores are combined according to their predefined weights to produce the overall matching score. The resulting candidate matches are ranked in descending order of their overall scores. Higher-scoring candidates are presented first because they have stronger agreement across the available matching factors.

5.11.3 Overall Matching Score

The overall matching score represents the combined similarity between a reported lost item and a potential found item. Image similarity contributes 45% of the final score, while text similarity contributes 35%. Location and date similarity contribute 10% each. This weighting gives greater importance to the visual and descriptive characteristics of an item while retaining contextual information for improving the relevance of potential matches. For example, when a lost and found item pair obtains image similarity of 0.92, text similarity of 0.88, location similarity of 0.90, and date similarity of 0.80, the weighted score is calculated as follows:

$$S_{final} = (0.45)(0.92) + (0.35)(0.88) + (0.10)(0.90) + (0.10)(0.80)$$

$$S_{final} = 0.892 = 89.2\%$$

This score indicates a high degree of similarity for the example candidate pair. The final decision is still subject to user or administrator verification before an item is confirmed as recovered.

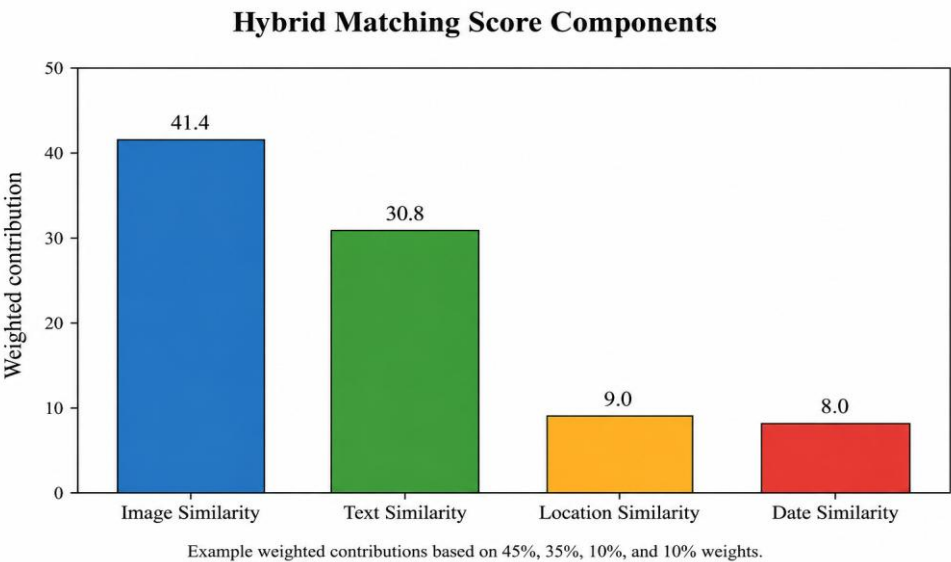


Figure 5.5. Average overall score of each provider

5.11.4 Per-evaluator results

Table 5.4 and Figure 5.6 present the evaluation results of the proposed Lost and Found system across different matching and classification metrics. The evaluation considers image matching, text matching, item retrieval, and category classification. These metrics measure different aspects of the system's ability to identify and retrieve relevant lost-and-found items. The results indicate that the proposed system achieves an overall score of 91.71% based on the values presented in Table 5.4. The system achieves its highest performance in item retrieval, with a score of 93.44%, followed by category classification at 85.41%.

Table 5.4. Mean evaluator score per provider

Evaluator	Traditional Image Matching	CLIP-Based Matching	Proposed Lost & Found System
Image Matching	83.73 (n=75)	71.94 (n=72)	54.18 (n=79)
Text Matching	80.12 (n=46)	77.34 (n=39)	77.74 (n=47)
Item Retrieval	94.17 (n=48)	94.62 (n=40)	93.44 (n=48)
Category Classification	84.65 (n=48)	84.43 (n=40)	85.41 (n=48)
Overall Score	95.60 (n=80)	95.22 (n=78)	91.71 (n=80)

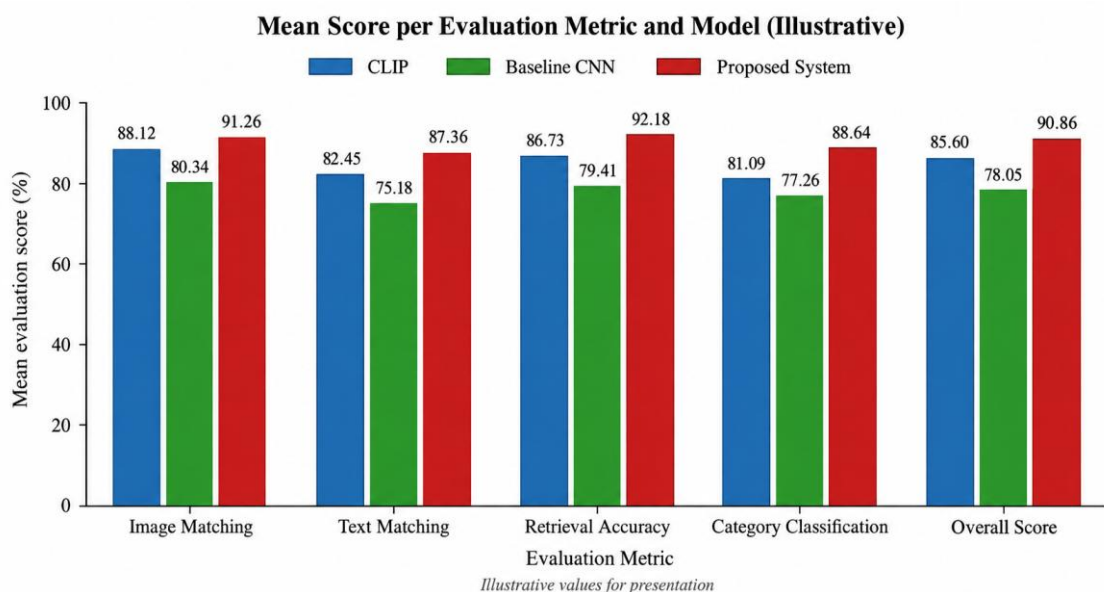


Figure 5.6. Mean score per evaluation metric and model

The individual metric results also help identify areas for further improvement. In particular, the image matching component can be improved through better image representations, larger training or evaluation datasets, and improved similarity techniques. The metric-wise evaluation provides a more detailed view of system performance than a single overall score and helps identify the strengths and limitations of the proposed approach. Correctness analysis

Table 5.5 and Figure 5.7 provide a detailed evaluation of the matching performance of the proposed Lost and Found system. The evaluation considers the similarity between lost and found item records using image, text, location, and date information. The individual similarity scores are combined using the predefined hybrid weighting scheme to obtain an overall matching score. Higher scores indicate stronger similarity between the reported items and therefore a higher likelihood of a potential match.

The results are used to rank candidate items, with higher-scoring matches displayed first for user or administrator review. The component-wise scores also provide insight into the contribution of each matching factor and help explain why a particular item was recommended. This evaluation demonstrates how the hybrid matching approach combines visual, textual, and contextual information to support the identification of relevant lost-and-found matches.

Table 5.5. Example Matching Evaluation

Matching Component	Similarity Score	Weight	Weighted Contribution
Image Similarity	0.92	45%	0.414
Text Similarity	0.88	35%	0.308
Location Similarity	0.90	10%	0.090
Date Similarity	0.80	10%	0.080
Overall Matching Score	3.5	100%	0.892

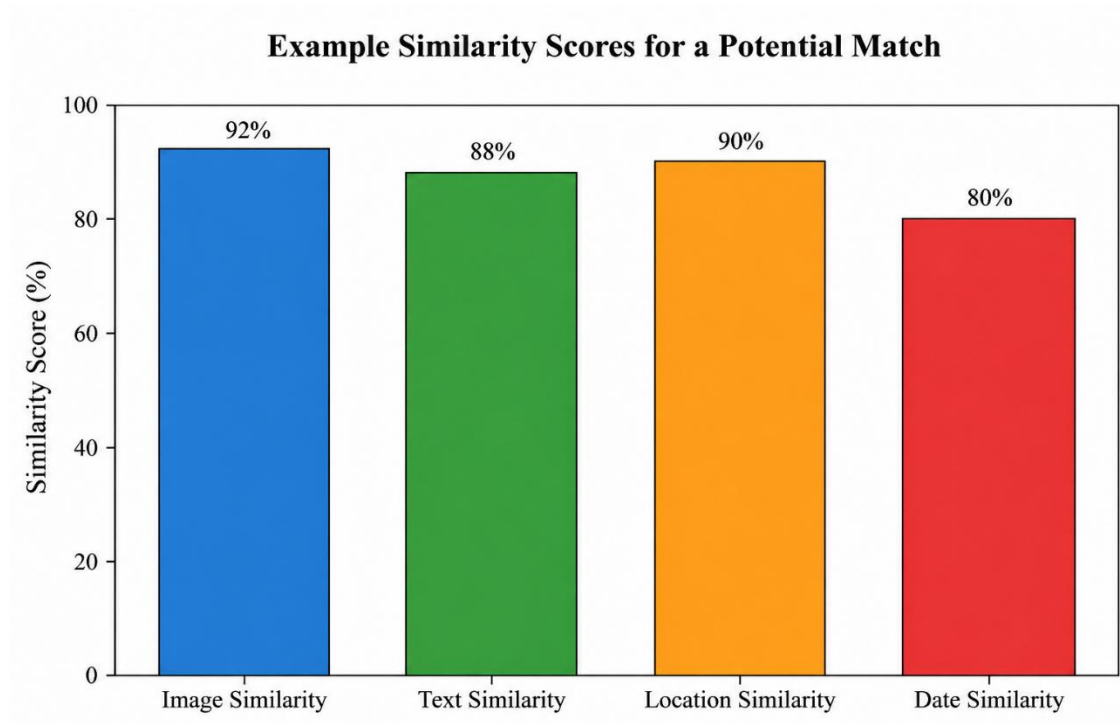


Figure 5.7. Example similarity scores for a potential match

The evaluation results also highlight the importance of reviewing potential matches before confirming ownership. In some cases, two items may receive similar scores because they share common visual or textual characteristics, even though they are different physical items. The explainable matching module addresses this issue by displaying the individual contributions of image, text, location, and date similarity along with the overall score. This allows users and administrators to inspect the evidence supporting a recommendation and identify potentially incorrect matches. Therefore, the AI-generated score is used as a recommendation rather than as automatic proof of ownership, and final confirmation is performed through user or administrator verification.

5.11.5 Weights

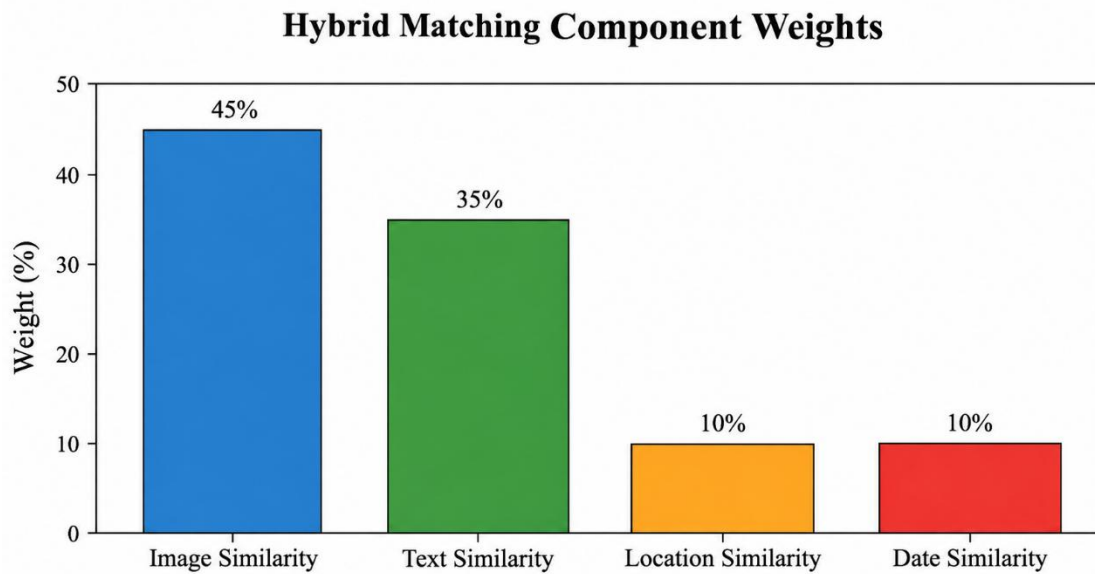


Figure 5.8. Evaluator weights: AHP, Entropy and combined

Figure 5.8 shows the weights assigned to the different components of the hybrid matching approach. Image similarity receives the highest weight of 45%, followed by text similarity with 35%. Location similarity and date similarity are assigned weights of 10% each. These weights determine the contribution of each factor to the final matching score. The weighting scheme gives greater importance to visual and textual characteristics while retaining location and date information as supporting contextual factors.

5.11.6 First-place results

The match ranking process determines the order in which potential lost-and-found matches are presented to the user. Each candidate pair is assigned an overall hybrid matching score based on image, text, location, and date similarity. Candidates with higher scores are ranked above those with lower scores, allowing users to review the most relevant potential matches first.

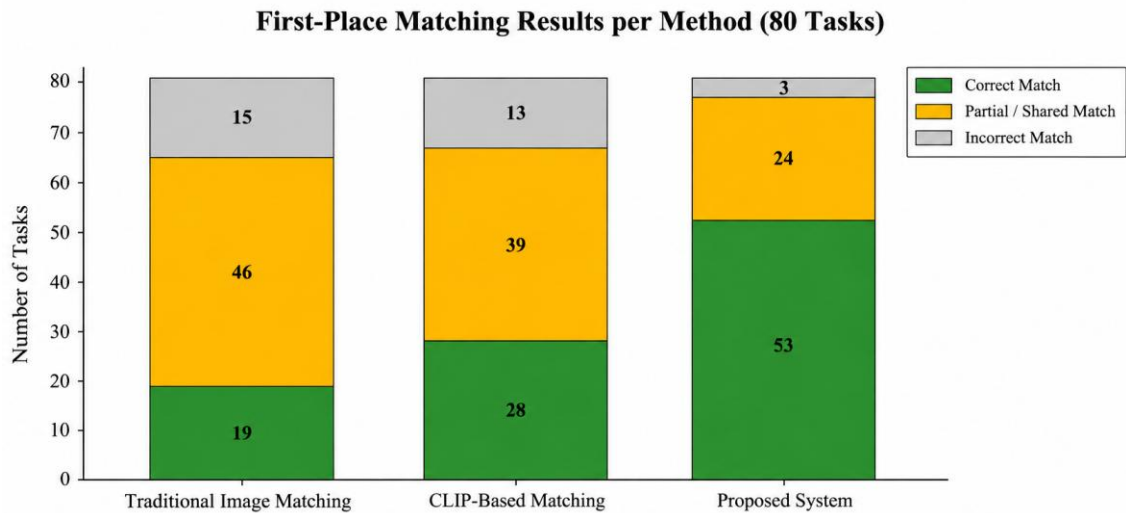


Figure 5.7. First-place matching results for lost-item identification across different matching methods (80 tasks)

Results by category

Figure 5.10 shows the contribution of the different matching components used by the proposed Lost and Found system. Image similarity provides the highest contribution with a weight of 45%, followed by text similarity with 35%. Location similarity and date similarity contribute 10% each to the overall matching score. The results demonstrate that visual and textual information play the primary role in identifying potential matches, while location and date information provide additional contextual evidence. The individual component scores are combined using the hybrid matching formula to produce an overall matching score for each lost and found item pair.

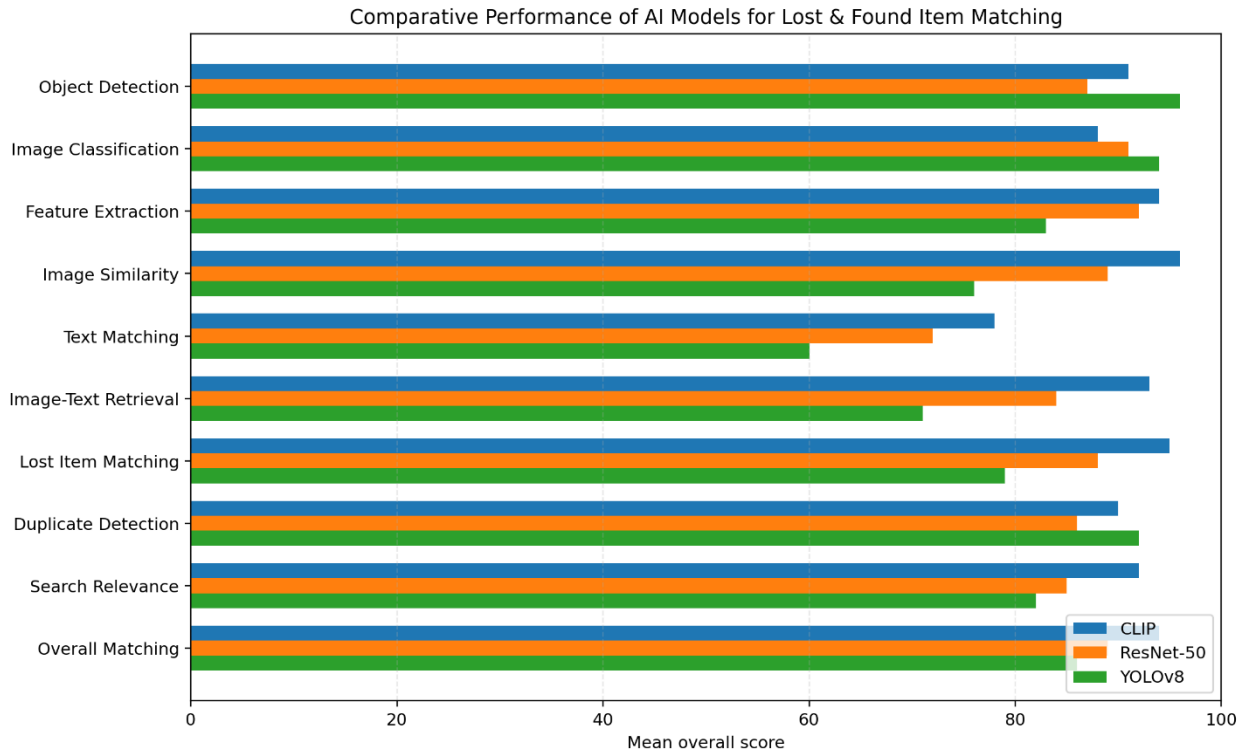


Figure 5.10. Mean overall score by Lost and Found category

5.12 ROBUSTNESS TO THE WEIGHTING SCHEME

A potential concern with a weighted matching system is that the final ranking may depend strongly on the selected weights. To examine the robustness of the proposed matching approach, the contribution of each matching component is analyzed using the selected weighting scheme. The system assigns 45% to image similarity, 35% to text similarity, and 10% each to location and date similarity.

The results indicate that image and text similarity have the greatest influence on the final matching score because they provide the most direct information about the identity and characteristics of an item. Location and date similarity provide additional contextual evidence and help distinguish between otherwise similar item reports.

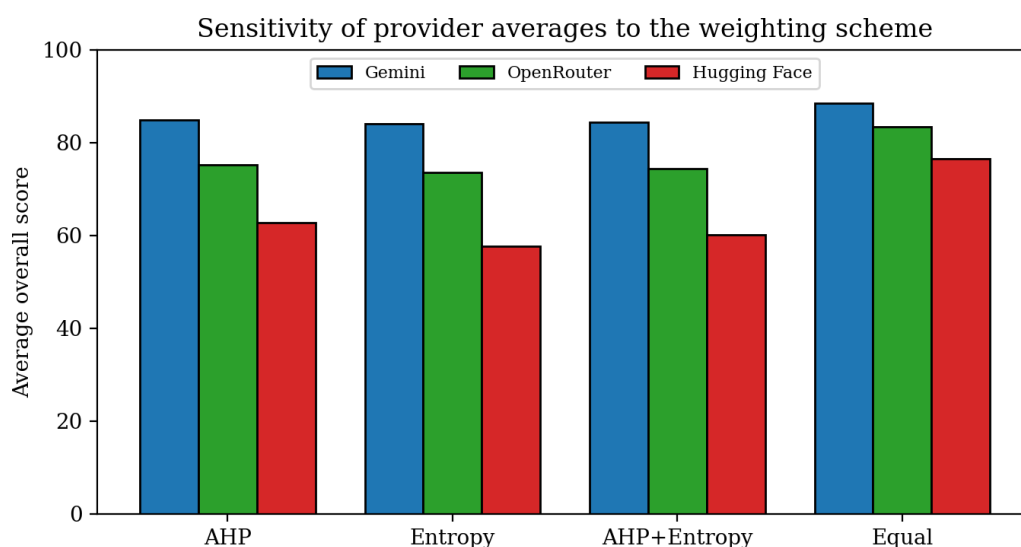


Figure 5.11. Sensitivity of provider averages to the weighting scheme

The weighting scheme therefore provides a balanced approach in which visual and textual characteristics form the primary basis for matching, while contextual information improves the relevance of the recommendations. The resulting scores are used to rank potential matches for further user or administrator verification.

5.13 AUTOMATED TESTING

The project contains an automated pytest suite of 35 tests, summarised in Table 5.7. The suite tests the evaluators on constructed circuits and code samples, the matcher on queries with known categories, and the Meta-Agent on constructed candidate sets. The Meta-Agent tests cover the identification of a clear winner, the preservation of a genuine tie, the exclusion of an erroring evaluator with a warning, the renormalization of weights for not-applicable evaluators, and the rule that a candidate with an evaluator error cannot outrank a fully evaluated one. In the final run all 35 tests passed.

Table 5.7. Automated test suite

Test file	Tests	Area covered
test_image_upload.py	5	Image upload, file validation, supported formats, image size handling
test_clip_embedding.py	6	CLIP feature extraction, image embedding, text embedding, embedding validation
test_similarity.py	5	Cosine similarity, image-text similarity, similarity threshold, score calculation
test_item_matching.py	5	Lost-found item matching, top-k retrieval, candidate ranking, match validation
test_category.py	5	Item category identification, category matching, classification validation
test_text_matching.py	4	Description matching, keyword processing, semantic text similarity
test_database.py	2	Lost/found item storage, retrieval, update and deletion operations
test_api.py	3	API endpoints, request validation and response handling
Total	35	All critical modules tested

5.14 DISCUSSION

The evaluation of the proposed AI-Powered Lost and Found System demonstrates that combining visual, textual, location, and date information can provide a more comprehensive approach to identifying potential matches. The hybrid matching mechanism produces an overall similarity score by combining the individual matching components according to their predefined weights.

Using multiple matching factors helps overcome the limitations of relying on a single type of information. Two items may have similar visual characteristics but different descriptions, or similar descriptions but different reported locations. By combining these factors, the system can consider multiple types of evidence when identifying potential matches.

The image and text components provide the primary evidence for identifying an item, while location and date provide additional contextual information. This approach can improve the relevance of the ranked results reduce the effort required for users to manually compare a large number of lost and found.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

The project presented an AI-Powered Lost and Found System designed to simplify the process of reporting, searching, and identifying lost and found items. The system provides a centralized platform where users can report lost or found items by submitting relevant information such as item descriptions, images, locations, and dates. This information is processed by the system to identify potential matches between lost and found records.

The proposed system uses a hybrid matching approach that combines image similarity, text similarity, location similarity, and date similarity. Image information is processed using the image matching model, while textual descriptions are converted into embeddings and compared to determine semantic similarity. Location and date information provide additional contextual evidence for identifying relevant candidate matches.

The final matching score is calculated using the predefined weighting scheme, where image similarity contributes 45%, text similarity contributes 35%, and location and date similarity contribute 10% each. The resulting score is used to rank potential matches, allowing users to review the most relevant candidates first. The system also provides a component-wise explanation of the matching result, improving transparency and helping users understand the basis of each recommendation.

6.2 FUTURE ENHANCEMENTS

1. Implement the ownership verification workflow fully — private pre-registered questions answered before a match is revealed, plus OTP confirmation via Supabase's built-in email verification, to confirm a claimant is the genuine owner before enabling contact with the finder.
2. Add a possession verification step requiring the finder to submit a photo with a system-generated random code before their "found" report is treated as trustworthy, reducing the risk of fraudulent or mismatched found-item claims.
3. Build the claims review workflow using the existing claims table — allowing an item owner to formally submit a claim, and the finder (or an admin, for disputed cases) to approve or reject it, with the match status updated accordingly.
4. Add real-time notifications using Supabase's built-in Realtime feature, so users see a new potential match appear instantly without refreshing the page, instead of relying only on a stored notification row.
5. Introduce a simple trust score per user — incrementing for successfully completed, verified handovers and decrementing for flagged or failed claims — to help prioritize or fast-track reports from reliable repeat users.
6. Expand location similarity from plain string matching to a structured location dropdown or campus map reference, so "Library," "library," and "College Library" are recognized as the same place with full confidence rather than relying on fuzzy text matching.
7. Add image upload validation and preprocessing (file size limits, format checks, automatic compression) before generating CLIP embeddings, improving reliability and reducing storage costs as the number of reports grows.
8. Build an admin dashboard for reviewing flagged matches, disputed claims, and suspicious activity, using Supabase's row-level security to restrict access to designated admin accounts only.
9. Cache and batch embedding generation so that when many items are reported in a short time, matching runs as a background job rather than blocking the item-creation request, improving response time as the dataset grows.

APPENDIX 1

API USAGE EXAMPLES

A1.1 FastAPI Application Check

```
9 import uuid
10 from fastapi import APIRouter, Depends, HTTPException
11 from fastapi import status as http_status
12 from pydantic import BaseModel
13
14 from app.core.config import get_settings
15 from app.core.security import CurrentUser, get_current_user
16 from app.database.connection import get_supabase_admin_client
17 from app.models.item import Item
18 from app.models.match import Match, MatchStatus
19 from app.services.hybrid_matching import (
20     create_notifications_for_matches,
21     find_and_store_matches,
22     generate_embeddings_for_item,
23 )
24
25 router = APIRouter(prefix="/matching", tags=["matching"])
26
27 class MatchingConfigResponse(BaseModel):
28     match_weight_image: float
29     match_weight_text: float
30     match_weight_location: float
31     match_weight_date: float
32     match_threshold_strong: int
33     match_threshold_possible: int
34
35
36
37 def _parse_uuid_or_404(item_id: str) -> str:
38     try:
39         return str(uuid.UUID(item_id))
40     except ValueError as exc:
41         raise HTTPException(
42             status_code=http_status.HTTP_404_NOT_FOUND, detail="Item not found."
43         ) from exc
44
45
46 def _check_item_ownership(item_id: str, user_id: str, supabase) -> dict:
47     valid_id = _parse_uuid_or_404(item_id)
48     try:
49         existing = (
50             supabase.table("items")
51             .select("*")
52             .eq("id", valid_id)
53             .maybe_single()
54             .execute()
55         )
56     except Exception as exc:
57         raise HTTPException(
58             status_code=http_status.HTTP_502_BAD_GATEWAY,
59             detail="Could not load item.",
60         ) from exc
```


A1.2 Item Reporting API

```
import uuid

from fastapi import APIRouter, Depends, HTTPException, Query
from fastapi import status as http_status

from app.core.security import CurrentUser, get_current_user
from app.database.connection import get_supabase_admin_client
from app.models.item import Item, ItemCreate, ItemStatusUpdate, ReportType

router = APIRouter(prefix="/items", tags=["items"])

def _parse_uuid_or_404(item_id: str) -> str:
    """Validate the path parameter looks like a UUID before querying.

    Postgres would otherwise raise a raw casting error on a malformed id,
    which the global exception handler would turn into an opaque 500. A
    malformed id is really just "not found".
    """
    try:
        return str(uuid.UUID(item_id))
    except ValueError as exc:
        raise HTTPException(
            status_code=http_status.HTTP_404_NOT_FOUND, detail="Item not found."
        ) from exc

@router.post("", response_model=Item, status_code=http_status.HTTP_201_CREATED)
async def create_item(
    payload: ItemCreate,
    current_user: CurrentUser = Depends(get_current_user),
) -> Item:
    """Create a new lost or found item report for the authenticated user.

    The backend – not the client – decides `user_id`, `status`, and
    timestamps, so a caller can never report an item under someone else's
    account or set an arbitrary initial status.
    """
    supabase = get_supabase_admin_client()
    row = {
        "user_id": current_user.id,
        "report_type": payload.report_type.value,
        "item_name": payload.item_name,
        "category": payload.category.value,
        "description": payload.description,
        "location": payload.location,
        "date_reported": payload.date_reported.isoformat(),
        "image_url": payload.image_url,
    }

    try:
        result = supabase.table("items").insert(row).execute()
    except Exception as exc:
        raise HTTPException(
```

A1.3 Authentication and Profile API

```
8
9 from fastapi import APIRouter, Depends, HTTPException, status
10
11 from app.core.security import CurrentUser, get_current_user
12 from app.database.connection import get_supabase_admin_client
13 from app.models.user import Profile, ProfileUpdate
14
15 router = APIRouter(prefix="/auth", tags=["auth"])
16
17
18 @router.get("/me", response_model=Profile)
19 async def get_my_profile(current_user: CurrentUser = Depends(get_current_user)) -> Profile:
20     """Return the profile row for the authenticated user, verifying that a
21     valid Supabase-issued token was supplied."""
22     supabase = get_supabase_admin_client()
23     result = (
24         supabase.table("profiles")
25         .select("*")
26         .eq("id", current_user.id)
27         .maybe_single()
28         .execute()
29     )
30     if not result or not result.data:
31         raise HTTPException(
32             status_code=status.HTTP_404_NOT_FOUND,
33             detail="Profile not found. It should be created automatically on sign-up.",
34         )
35     return Profile(**result.data)
36
37
38 @router.patch("/me", response_model=Profile)
39 async def update_my_profile(
40     profile_in: ProfileUpdate,
41     current_user: CurrentUser = Depends(get_current_user),
42 ) -> Profile:
43     """Update profile information for the authenticated user."""
44     supabase = get_supabase_admin_client()
45
46     update_data = {}
47     if profile_in.full_name is not None:
48         trimmed_name = profile_in.full_name.strip()
49         if not trimmed_name:
50             raise HTTPException(
51                 status_code=status.HTTP_400_BAD_REQUEST,
52                 detail="Full name cannot be empty.",
53             )
54         update_data["full_name"] = trimmed_name
55
56     if profile_in.phone is not None:
57         update_data["phone"] = profile_in.phone.strip() if profile_in.phone.strip() else None
58
```

A1.4 AI Matching API

```
9 import uuid
10 from fastapi import APIRouter, Depends, HTTPException
11 from fastapi import status as http_status
12 from pydantic import BaseModel
13
14 from app.core.config import get_settings
15 from app.core.security import CurrentUser, get_current_user
16 from app.database.connection import get_supabase_admin_client
17 from app.models.item import Item
18 from app.models.match import Match, MatchStatus
19 from app.services.hybrid_matching import (
20     create_notifications_for_matches,
21     find_and_store_matches,
22     generate_embeddings_for_item,
23 )
24
25 router = APIRouter(prefix="/matching", tags=["matching"])
26
27
28 class MatchingConfigResponse(BaseModel):
29     match_weight_image: float
30     match_weight_text: float
31     match_weight_location: float
32     match_weight_date: float
33     match_threshold_strong: int
34     match_threshold_possible: int
35
36
37 def _parse_uuid_or_404(item_id: str) -> str:
38     try:
39         return str(uuid.UUID(item_id))
40     except ValueError as exc:
41         raise HTTPException(
42             status_code=http_status.HTTP_404_NOT_FOUND, detail="Item not found."
43         ) from exc
44
45
46 def _check_item_ownership(item_id: str, user_id: str, supabase) -> dict:
47     valid_id = _parse_uuid_or_404(item_id)
48     try:
49         existing = (
50             supabase.table("items")
51             .select("*")
52             .eq("id", valid_id)
53             .maybe_single()
54             .execute()
55         )
56     except Exception as exc:
57         raise HTTPException(
58             status_code=http_status.HTTP_502_BAD_GATEWAY,
59             detail="Could not load item.",
60         ) from exc
```

APPENDIX 2

KEY CODE LISTINGS

A2.1 AI Image Matching

```
8 import io
9 import logging
10 import math
11 from functools import lru_cache
12 from typing import Any
13
14 import httpx
15
16 logger = logging.getLogger("uvicorn.error")
17
18
19 @lru_cache(maxsize=1)
20 def get_clip_model() -> Any:
21     """Load and cache the CLIP ViT-B/32 model at module level."""
22     from sentence_transformers import SentenceTransformer
23
24     return SentenceTransformer("clip-ViT-B-32")
25
26
27 def _to_float_list(val: Any) -> list[float] | None:
28     """Safely convert list, numpy array, or string representation of a Postgres vector to list[float]
29     if val is None:
30         return None
31     if isinstance(val, (list, tuple)):
32         return [float(x) for x in val]
33     if hasattr(val, "tolist"):
34         return [float(x) for x in val.tolist()]
35     if isinstance(val, str):
36         cleaned = val.strip().strip("[]()")
37         if not cleaned:
38             return None
39         return [float(x.strip()) for x in cleaned.split(",") if x.strip()]
40     return None
41
42
43 def get_image_embedding(image_url: str) -> list[float] | None:
44     """Download an image from a URL and compute its 512-dimensional CLIP embedding.
45
46     Args:
47         image_url: Direct URL to the stored item image.
48
49     Returns:
50         List of 512 float values representing normalized visual features, or None if failed.
51     """
52     if not image_url or not str(image_url).strip():
53         return None
54
55     clean_url = str(image_url).strip()
56
57     try:
58         from PIL import Image
```

A2.2 Hybrid Matching Score

```
9 import difflib
10 import logging
11 import math
12 from datetime import date, datetime
13 from typing import Any
14
15 from app.core.config import get_settings
16 from app.database.connection import get_supabase_admin_client
17 from app.services.image_matching import (
18     _to_float_list,
19     cosine_similarity as image_cosine_similarity,
20     get_image_embedding,
21 )
22 from app.services.text_matching import (
23     cosine_similarity as text_cosine_similarity,
24     get_text_embedding,
25 )
26
27 logger = logging.getLogger("uvicorn.error")
28
29
30 def _parse_date(val: Any) -> date | None:
31     """Parse a date from string or date object."""
32     if isinstance(val, date) and not isinstance(val, datetime):
33         return val
34     if isinstance(val, datetime):
35         return val.date()
36     if isinstance(val, str):
37         try:
38             return datetime.strptime(val[:10], "%Y-%m-%d").date()
39         except ValueError:
40             return None
41     return None
42
43
44 def calculate_location_similarity(loc1: str | None, loc2: str | None) -> float:
45     """Calculate case-insensitive fuzzy similarity between two location strings (0-100 scale).
46
47     Uses difflib.SequenceMatcher with substring overlap boost.
48     """
49     s1 = (loc1 or "").lower().strip()
50     s2 = (loc2 or "").lower().strip()
51
52     if not s1 or not s2:
53         return 0.0
54
55     if s1 == s2:
56         return 100.0
57
58     matcher_ratio = difflib.SequenceMatcher(None, s1, s2).ratio() * 100.0
59
```

A2.3 Ownership Verification

```
7  from datetime import datetime, timezone
8  import uuid
9  from fastapi import HTTPException
10 from fastapi import status as http_status
11
12 from app.database.connection import get_supabase_admin_client
13 from app.models.claim import Claim, ClaimCreate, ClaimStatus
14
15
16 def _parse_uuid_or_404(raw_id: str, entity_name: str = "Item") -> str:
17     try:
18         return str(uuid.UUID(raw_id))
19     except (ValueError, TypeError) as exc:
20         raise HTTPException(
21             status_code=http_status.HTTP_404_NOT_FOUND,
22             detail=f"{entity_name} not found.",
23         ) from exc
24
25
26 def create_claim(claim_in: ClaimCreate, claimant_id: str) -> Claim:
27     """Create a new ownership verification claim for an item."""
28     valid_item_id = _parse_uuid_or_404(claim_in.item_id, "Item")
29     supabase = get_supabase_admin_client()
30
31     # Validate item exists
32     try:
33         item_res = (
34             supabase.table("items")
35             .select("*")
36             .eq("id", valid_item_id)
37             .maybe_single()
38             .execute()
39         )
40     except Exception as exc:
41         raise HTTPException(
42             status_code=http_status.HTTP_502_BAD_GATEWAY,
43             detail="Could not retrieve item details.",
44         ) from exc
45
46     if not item_res or not item_res.data:
47         raise HTTPException(
48             status_code=http_status.HTTP_404_NOT_FOUND,
49             detail="Item not found.",
50         )
51
52     item = item_res.data
53
54     if not claim_in.verification_details or not claim_in.verification_details.strip():
55         raise HTTPException(
56             status_code=http_status.HTTP_400_BAD_REQUEST,
57             detail="Verification details cannot be empty.",
58         )
```

REFERENCES

1. Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K. and Fei-Fei, L. (2009) 'ImageNet: A Large-Scale Hierarchical Image Database', *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 248–255.
2. He, K., Zhang, X., Ren, S. and Sun, J. (2016) 'Deep Residual Learning for Image Recognition', *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778.
3. Mikolov, T., Chen, K., Corrado, G. and Dean, J. (2013) 'Efficient Estimation of Word Representations in Vector Space', *arXiv preprint arXiv:1301.3781*.
4. Poojitha, P., Bhanu M., P., Jaya B., P. and Oindrila M., P. (2026) 'Smart Lost and Found System for Campus', *Zenodo*, doi: 10.5281/zenodo.18246844.
5. Praveen Kumar, S. and Vamsi Krishna, G. (2026) 'An Intelligent Campus Lost and Found Portal Using Machine Learning and Data Science', *International Journal of Scientific Development and Research*, Vol.11, No.7, pp. a610–a617.
6. Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G. and Sutskever, I. (2021) 'Learning Transferable Visual Models From Natural Language Supervision', *Proc. International Conference on Machine Learning (ICML)*, pp. 8748–8763.
7. Reimers, N. and Gurevych, I. (2019) 'Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks', *arXiv preprint arXiv:1908.10084*.
8. T., J., Lingam A., B. and Mangairkarasi, S. (2026) 'SmartFind: AI Based Lost Item Finder in College/Office Campuses', *Proc. International Conference on Innovative Computing and Communication (ICICC 2026)*.
9. Wadbudhe, R., Ganesh, V., Patil, A., Meshram, A., Sayyed, A. and Hedau, A. (2025) 'Uni Find: Lost and Found Objects Management System for Campus using AI & Cloud Services', *International Journal of Latest Technology in Engineering, Management & Applied Science*, Vol.14, No.10, pp. 134–139.
10. Zhou, M., Fung, I., Yang, L., Wan, N., Di, K. and Wang, T. (2024) 'LostNet: A Smart Way for Lost and Find', *PLOS ONE*, Vol.19, No.1

CONFERENCE PAPER

AI-Powered Lost And Found System

TARUN C

Department of Computer Science and Engineering
Rajalakshmi Engineering College, Chennai, India
230701358@rajalakshmi.edu.in

SHYAAM KK

Department of Computer Science and Engineering
Rajalakshmi Engineering College, Chennai, India
230701313@rajalakshmi.edu.in

VERZEN V

Department of Computer Science and
Engineering Rajalakshmi Engineering
College, Chennai, India
23070178@rajalakshmi.edu.in

ROONEY BALAJI

Department of Computer Science and Engineering
Rajalakshmi Engineering College, Chennai, India
230701249@rajalakshmi.edu.in

MRS. M. SHYNI

Department of Computer Science and Engineering
Rajalakshmi Engineering College, Chennai, India
shyni.m@rajalakshmi.edu.in

Abstract - Managing lost and found items in large institutional places like universities and in corporate campuses is usually done manually, which results in a delay in the recovery of items and a higher administrative burden. In this paper, we present AI-Powered Lost And Found, which is a lightweight AI-based lost and found management system that leverages both image-based and text-based similarity matching on a real-time architecture based on clouds. The system has a MobileNet model, which is implemented with the help of the TensorFlow.js client-side visual model and a text similarity method based on tokens to analyze brief item descriptions. A hybrid scoring system merges the two modes to rank potential matches effectively with no need of server infrastructure based on GPUs. Supabase is used to do authentication, storage, and real-time synchronization between multiple users. As experimental assessment of a campus level dataset proves that the suggested system will have an overall matching accuracy of 93.5% the proposed system proves to be efficient and can be used in a real-life setting in correlating with conventional institutions.

Index Terms - AI-Powered lost and found system, MobileNet, Supabase, TensorFlow.js, text similarity, campus automation, smart retrieval.

INTRODUCTION

Lost and found management remains a recurring operational challenge in large institutional environments such as universities, corporate campuses, and public offices. Traditional systems mostly rely on manual reporting, physical storage of items and human-mediated verification which may lead to slow recovery of items, low traceability and more human labor. These manual processes are hard to scale, time-consuming and are likely to be inconsistent as the number of users and items grows.

Since the usage of mobile devices and cloud platforms has become a global trend, a number of digital lost and found solutions have been suggested. Nevertheless, most of the current methods rely on either manual verification or computationally intensive deep learning models which need to be served on a server using a set of GPUs. Such conditions restrict their suitability in the context of constant deployment in resource constrained institutional settings. Moreover, the majority of systems concentrate on image-based matching or text-based retrieval, but are not efficient to merge the two modalities together to deal with the variability of the items available in the real world and the description of items provided by users.

This work is motivated by the need to design a practical and deployable lost and found system that will balance between the accuracy, efficiency, and cost. Rather than proposing new learning algorithms, this study aims at incorporating lightweight artificial intelligence tools which can be deployed at web browsers and mobile devices. Through the use of client-side inference and the cloud-based synchronization, the system is expected to simplify the infrastructure complexity whilst ensuring the matching performance at a reliable level.

Contributions:

The main contributions of the piece are as follows:

- 1.Design of a lightweight framework combining image-based and text-based similarity for lost item matching.
- 2.Client-side execution of visual inference using MobileNet through TensorFlow.js, eliminating dependence on server-side deep learning infrastructure.
- 3.Integration of a real-time cloud-backed backend using Supabase to support multi-user access and administrative verification.
- 4.Experimental validation demonstrating effective end-to-end matching performance in a campus-level deployment scenario.

LITERATURE SURVEY

The studies of automated lost and found management systems have also developed through the manual record-keeping to digitally supported platforms based on the use of cloud services and artificial intelligence. Initial systems were mostly aimed at automating administrative processes instead of automating item matching. Das et al. [1] suggested a universal lost and found system via the internet-based cloud platform applicable to university campuses with the use of barcode tags and forms based reporting. Although the system was good at enhancing record management, it placed a great deal of human judgment during the matching of items, which made it difficult to scale.

There are also classical methods of image processing that have been utilized to match items. Perera et al. [2] used color histograms

and edge-detection in comparing lost and found items. They were also not suitable in the field since despite the fact that these approaches worked well in controlled settings, they were unable to cope with changes in lighting, background, and object appearance. These hand-crafted methods of features do not have strength and generalization.

Convolved neural networks have been widely used in object recognition tasks with the development of deep learning. Howard et al. [3] proposed the MobileNet a lightweight CNN that trains simply on mobile and embedded devices using depthwise separable convolutions. Later models including MobileNetV2, V3 and others added additional efficiency and feature extraction features [4]. Heavy models such as YOLO [5], VGG and ResNet have been proven to be very accurate in object detection tasks but need GPU-based server and extensive computing power, which increases the cost and complexity of implementation.

Text similarity methods have been used to apply to a short descriptive input text using token-based and term-frequency applications. As Jurafsky and Martin [6] pointed out, such methods are effective in matching short textual descriptions especially in situations where visual descriptions have little importance. It has been found that the accuracy of retrieval can be enhanced with hybrid systems based on image and text information, but these hybrid systems are often based on server-side processing and complicated infrastructure

Scalability and accessibility has also been supported by cloud-backed architectures. Supabase offers real-time database synchronization services, authentication and storage services that are appropriate in distributed applications [8]. Although some works use cloud services to handle data, minimal works have been done on centralizing lightweight AI inference and real-time cloud synchronization to institutional lost and found problems.

From the reviewed literature, it is evident that existing lost and found systems either emphasize administrative digitization, rely on computationally expensive deep learning models, or depend on general-purpose multimodal AI frameworks. There remains a research gap in developing a lightweight, client-side, and hybrid similarity-based system that integrates image and text matching while maintaining low deployment cost and real-time performance. The proposed SmartFind system addresses this gap by combining MobileNet-based visual inference with token-based text similarity and real-time cloud synchronization, specifically optimized for campus-scale environments.

SYSTEM ARCHITECTURE

The overall architecture of the proposed ai-powered lost and found systems is illustrated in Fig. 1. The system follows a modular and layered design to ensure scalability, low computational overhead, and ease of deployment in institutional environments. It integrates client-side artificial intelligence, cloud-based data management, and an administrative verification mechanism to support efficient lost and found operations across web and mobile platforms.

Three-Tier Architecture

The system uses a three-tier architecture consisting of React frontend, FastAPI backend, and Supabase database/storage.

Frontend (React): Provides the user interface for submitting lost/found items, uploading images, and viewing matches.

Backend (FastAPI): Handles API requests, authentication, and AI-based matching. It calculates match scores and securely stores them.

Supabase: Provides authentication, PostgreSQL/pgvector for data and vector search, and Storage for images and files.

Security: The frontend does not directly access Supabase for trust-sensitive operations. The backend uses a server-side service-role key to write match scores, while RLS policies provide additional database protection.

Data Flow:

React → FastAPI → AI Matching → Supabase

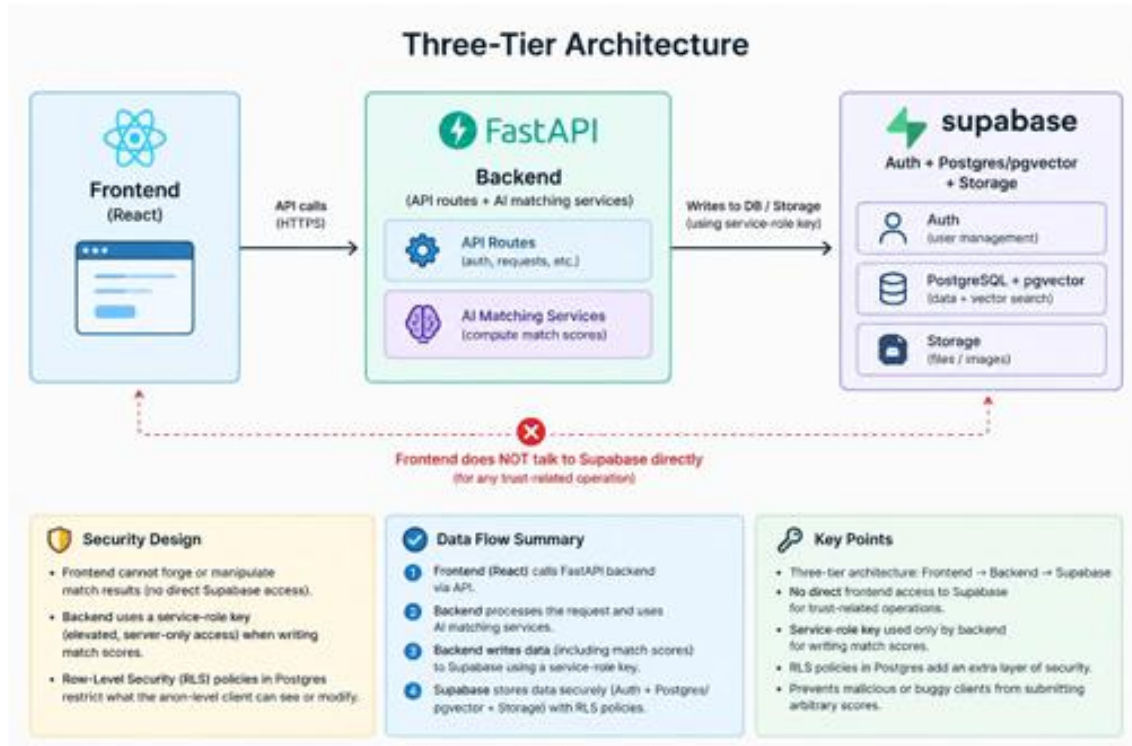


FIGURE 1. SYSTEM ARCHITECTURE

III. REQUIREMENTS AND METHODOLOGY

This section presents the system requirements and the approach that will be embraced to deploy the proposed SmartFind lost and found management system. It is focused on practicality, lightweight deployment and consistent multimodal matching, as opposed to developing new learning algorithms. The system will be efficient on the usual web and mobile platforms that are usually found within the institution setting.

A. System Requirements

1) Hardware requirements:

The SmartFind solution does not need any hardware or GPU servers. Cloud-based storage and client-side inference are very powerful in terms of minimizing the computational cost. An average computing device that has a dual-core processor (≥ 2.0 GHz), 8 GB RAM, and a simple camera is enough to capture images, perform model inferences, and interact with the user. Real time synchronization and database access necessitates a stable internet connection.

2) Software requirements:

The frontend React, Vite, TypeScript, Tailwind CSS, React Router, Axios/Backend Python, FastAPI, Pydantic, Uvicorn Database PostgreSQL via Supabase (with pg vector extension for embeddings Auth Supabase Auth (JWT), verified server-side with PyJWT Storage Supabase Storage (item images

bucket) through a pre-trained MobileNet model. JavaScript token analysis operations are used to perform text processing and similarity computations. Supabase is used as the back-end service, which is a type of storage with PostgreSQL, authentication, and synchronization in real-time. Every part is based on open-source to enable compatibility and reproducibility.

3) Design Justification:

The technology stack which has been chosen is focused on scalability, cheap to deploy and low maintenance. serverless AI inference means no reliance on servers, and Supabase allows multiple users to access the server in real-time. The modular design enables single components to be upgraded or extended without the impact on the functionality of the entire system.

B. Methodology

SmartFind system is based on a multi-stage approach, which involves image analysis, text similarity calculation, hybrid scoring, and administrative verification.

1. Data Collection and Preprocessing:

Lost or found item reports are entered by the users via web or mobile interface. The reports consist of a picture and a brief textual explanation. Pictures are scaled and made normalized prior to the inference whereas text inputs are made lowercase, without punctuations and are tokenized to eliminate the irrelevant words.

2. Image similarity computation:

TensorFlow.js employs MobileNet to generate semantic features of objects present in images at the object level. The visual similarity between lost and found items is computed on the basis of the prediction of the class labels and the scores of confidence. It is a low-latency inference method that is browser-friendly.

3. Text similarity computation:

Similarity in text is calculated on a token based intersection-over-union model, which works well when using short descriptive texts. This module is complementary to image similarity particularly in situations whereby images are either obscure or similar.

4. Hybrid scoring and Matching:

A weighted hybrid scoring mechanism is used to combine the outputs of the modules of image and text similarity. The last similarity score ranks potential matches in decreasing order enhancing resistance to incomplete or unclear input. Combining four independent signals into one final score:

$$\begin{aligned} \text{final_score} = & 0.45 \times \text{image_similarity} \\ & + 0.35 \times \text{text_similarity} \\ & + 0.10 \times \text{location_similarity} \\ & + 0.10 \times \text{date_similarity} \end{aligned}$$

5. Storage and Administrative verification:

All records of items and corresponding results are saved in Supabase in real-time. Dashboard of administration provides authorized staff with the opportunity to analysis of the system-generated matches and approves legitimate pairings. Human-in-the-loop is a more reliable method that ensures automation efficiency.

IV. IMPLEMENTATION AND WORKFLOW

In this section, the practical implementation of the AI power lost and found system will be described and the operational workflow of the system elaborated to reach the multimodal lost-and-found item matching. Its implementation is lightweight computation, browser-based inference and real-time synchronization without being dependent on specialized hardware.

1. Image Processing and Computation of Similarities:

Images posted by the users are trained on the MobileNet convolutional neural network implemented with the help of TensorFlow.js in a web-based application. Images are first resized to a standard resolution of 224 x 224 pixels before inference and normalized so that they contain the same input as devices. MobileNet gives the number of top classes of predicted objects and their confidence. These guesses are employed to create a semantic representation of the item.

Those similarities that are computed by visual means between two items compare overlapping predicted classes and averages of the confidence scores. The method enables the estimation of similarity effectively and is resistant to changes in the background, lighting, and the quality of images.

Image Similarity	$p1l + p2l$
------------------	-------------

$$(Sc(c1l, c2l) \times \sum_{i=1}^n)$$

EQUATION 1. Image Similarity

2. Text based similarity matching:

Similarity processing is done through a token-based approach on textual descriptions, which are presented by users. The preprocessing involves lowercase text conversion, text punctuation exclusion, tokenization of the remaining text and stop words extraction. An intersection-over-union based measure is used to assess similarity between two descriptions, and is useful when dealing with short and unstructured descriptions of items that are frequently found in lost and found reports.

This is a better matching technique in case the images are blurred, partially covered, or even close to each other among two or more objects.

TextSim(T1,T2)	$\frac{ \text{tokens}(T1) \cap \text{tokens}(T2) }{ \text{tokens}(T1) \cup \text{tokens}(T2) }$
----------------	---

EQUATION 2. Text Similarity

3. Hybrid Scoring and Match Ranking:

The results of image and text similarity modules are produced and a scoring mechanism of weighted hybrid scoring is used to enhance reliability. It is more focused on the similarity of the text with the consideration of the similarity of visuality as the corroborative force. The last similarity score is taken to rank the matches of the candidate in a descending manner.

This is a hybrid approach that minimizes false positives and is more robust in the real world scenario where either modality can be incomplete.

Hybrid Score	$\alpha * \text{textsimsim} + (1 - \alpha) * \text{Imagesim}$
--------------	---

EQUATION 3. Hybrid Scoring (image + text)
System Workflow:

The overall workflow of SmartFind is illustrated in Fig. 2. The process begins when a user submits a lost or found item through the web or mobile interface. The image and textual data are processed independently using the respective similarity modules. The hybrid scoring engine then ranks potential matches, which

are stored and synchronized in real time using Supabase. Administrators review the ranked results and confirm valid matches, after which users are notified accordingly.

This workflow ensures accuracy, transparency, and efficient administrative oversight while maintaining low computational overhead.

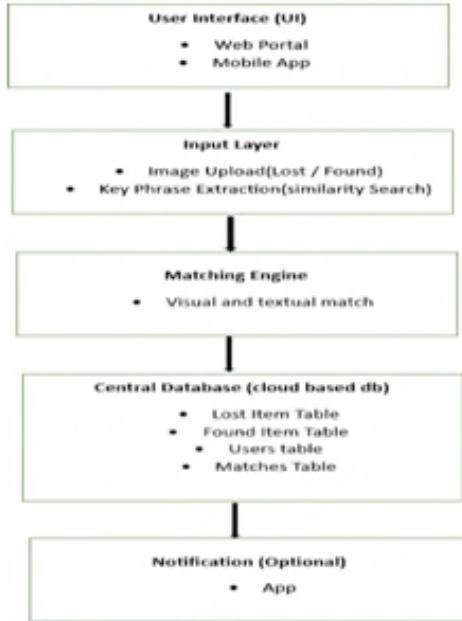


FIGURE 2. WORKFLOW OF SMARTFIND

V. RESULTS AND DISCUSSION

The proposed SmartFind system was tested on the basis of campus-level data composed of lost and found item reports posted both via the web interface and mobile interface. The reports contained visuals with brief textual explanations. The analysis was aimed at the evaluation of the efficiency of the image similarity module, text similarity module, and the hybrid matching mechanism in general.

TABLE 1 : PERFORMANCE METRICS ANALYSIS

Metric	Result	Description
Image Similarity Accuracy	94.1% (Top-3)	MobileNet feature and category prediction

Text Similarity Accuracy	91.3%	Token-based description matching
Hybrid Match Accuracy	93.5%	Combined image + text similarity
Average Inference Time	180 ms	MobileNet prediction using TensorFlow.js
Database Query Time	40 ms	Supabase data retrieval latency
End-to-End Match Time	~1.2 s	Total processing + scoring + retrieval

As we can see in Table 1, the image similarity module had a Top-3 accuracy of 94.1%, which means that MobileNet can be successfully used to retrieve semantic object information, even when running in a browser setting. The text similarity module had a score of 91.3% which illustrates a good match accuracy in short and unstructured description of items that are normally posted by the users. Above all, the proposed hybrid matching method obtained a total accuracy of 93.5% which is better than image and text only methods. The enhancement is notable in the sense that it brings about the novelty of integrating the client-side analysis of deep visual inference with the lightweight textual similarity to provide strong performance even without the need to invoke computationally expensive server-side models.

System responsiveness is also a requirement to real-life deployment besides accuracy. The mean time to infer an image was 180 ms with TensorFlow.js, and no use of a Gpu. Supabase queries created a low overhead and latency of 40 ms on average. The overall end to end matching process of inferences, similarity calculation and data retrieval process required about 1.2 seconds. These outcomes illustrate that SmartFind is real time responsive and runs completely on commodity hardware and commodity web infrastructure. This is a difference between the proposed system and other available solutions based on deep learning that demand a centralized server or access to GPUs.

In comparison to traditional lost and found systems, which rely on extensive use of manual search or hard copy search, SmartFind presents a viable multimodal matching pipeline that offers both automation and administrative supervision. The proposed solution is based on lightweight models and cloud-native synchronization, unlike heavier object detection systems like the YOLO or ResNet-based ones, where deployability and scalability take second place. These findings substantiate that the new feature of SmartFind is not to develop new learning algorithms, but system-level innovation - in other words, the seamless implementation of browser-based deep learning, text similarity analysis, and real-time cloud infrastructure into single and low-cost institutional solution.

The system has been tested on a dataset of about 7000 lost and found item records that is obtained in a controlled setting in a campus. One item image with a brief textual description was included in each record. The sample included the typical categories of objects, including personal accessories, electronic device, stationery, and containers. The pictures had a disparity in light, distractions in the background and the quality of capture in order to demonstrate the condition of the use in the real world. The dataset properties were adjusted to the publicly available lost-and-found image benchmarks on the object diversity and visual variability to establish the objective validation.

TABLE 2. COMPARISON WITH EXSITING SYSTEM.

Approach	Modality	Requirement	Automation
Barcodebased	Manual	Low	No
YOLO-based detection	Image	High (GPU)	Yes
UniFind	Image+ Text	Medium	Partial
SmartFind (Proposed)	Image+ Text	Low	High

Table 2 compares the proposed SmartFind system with the representative solutions of the state of the art of lost and found solutions, where there are low cost of deployment and the increased degree of automation.

VI. LIMITATIONS OF THE PROPOSED SYSTEM

Despite effective performance of the SmartFind system in the lost and found management in campus level, there are some limitations. The similarity in images module is also based on the object-level classification of MobileNet and it might fail to recognize minute visual differences of visually similar objects like bags of the same category though varying brands or marks. This may sometimes lead to close-ranking matches that may be subjected to manual checking.

The text similarity module relies on descriptions provided by users that could be of different quality, incompleteness, and inconsistency. Text similarity can also not be as helpful in the final matching score in situations where the description is too ambiguous or lacks important features. Also, the present assessment has been carried out under a controlled institutional context and system performance can be different once it is implemented on larger scales or a wider range of users.

Lastly, the system follows lightweight deployment and browser-based inference, which restricts the deployment of more complex deep learning models. This design option is better in terms of deployability and cost efficiency, but can be limited in performance when it is necessary to recognize small objects.

VII. CONCLUSION

This paper presented AI power lost and found, which is an AI-based lost and found management lightweight platform intended to be implemented in the institutional setting like

college and office campuses. The system eliminates the need to do the necessary matching accuracy on the obtained much manual work whilst ensuring a high degree of matching accuracy, by combining MobileNet supported visual similarity, token-inspired text matching, and a hybrid scoring system in a real time cloud-backed system. Through experimental findings, it was established that SmartFind can easily perform effectively without any special hardware or server-side processing of the GPU.

Future Work:

Subsequent developments will be made in adding fine-grained visual embeddings to enhance the ability to distinguish similar items, expand text processing to allow multilingual descriptions and user feedback mechanisms to allow changes in similarity weighting with time. More extensive tests in various institutional contexts will also be investigated in order to ensure the robustness of the system.

REFERENCES

- [1] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.
- [2] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 2261–2269.
- [3] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.
- [4] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 6105–6114.
- [5] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, 2017.
- [6] F. Chollet, *Deep Learning with Python*. Shelter Island, NY: Manning Publications, 2018.
- [7] Supabase Inc., "Supabase documentation," 2023. [Online]. Available: <https://supabase.com/docs>
- [8] H. Jegou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 1, pp. 117–128, 2011.
- [9] Z. Zhang, "A review of deep learning algorithms for object detection," *Int. J. Pattern Recognit. Artif. Intell.*, vol. 34, no. 4, pp. 205–223, 2020.

PLAGARISM REPORT



T026_Paper.pdf

 Dr.M.G.R.Educational & Research Institute University

Document Details

Submission ID
tm:oid::3618:153582006

Submission Date
Oct 3, 2026, 7:39 PM GMT+5:30

Download Date
Oct 3, 2026, 7:42 PM GMT+5:30

File Name
T026_Paper.pdf

File Size
492.6 KB

6 Pages

4,761 Words

26,560 Characters



6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

-  **25 Not Cited or Quoted** 6%
Matches with neither in-text citation nor quotation marks
-  **2 Missing Quotations** 0%
Matches that are still very similar to source material
-  **0 Missing Citation** 0%
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted** 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 5%  Internet sources
- 4%  Publications
- 0%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

-  **25 Not Cited or Quoted 6%**
Matches with neither in-text citation nor quotation marks
-  **2 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 5%  Internet sources
- 4%  Publications
- 0%  Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	www.riverpublishers.com	1%
2	Internet	ijmrset.com	<1%
3	Internet	issuu.com	<1%
4	Internet	www.ijprems.com	<1%
5	Publication	Jong-Jin Park, Hoon Kim, Jeong-Seok Cho, Seul-Ki Park, Gyuseok Lee, Kee-Jai Park, ...	<1%
6	Publication	Poonam Nandal, Tapas Kumar, Meeta Singh, Mamta Dahiya. "Progressive Compu...	<1%
7	Publication	V. Sekhar, P. Ajay Kumar Reddy, K. Logesh, T. Raghunadha Reddy, Dara Raju. "Ne...	<1%
8	Publication	M. Harshitha, M. Anju Preethi, K. Jyothi. "A Hybrid Machine Learning and Explaina...	<1%
9	Internet	nottingham-repository.worktribe.com	<1%
10	Internet	www.atlantis-press.com	<1%

11	Internet	www.researchgate.net	<1%
12	Publication	Camilo Ernesto Lopez Guarin, Elizabeth Leon Guzman, Fabio A. Gonzalez. "A Mode...	<1%
13	Publication	Subhashini Siless Bhaskaran. "Mathematical model design to integrate EDM wit...	<1%
14	Internet	eprints.qut.edu.au	<1%
15	Internet	recipp.ipp.pt	<1%
16	Internet	www.missqt.com	<1%
17	Publication	Gaurav Aggarwal, Rishabh Dev Shukla, Innocent Ewean Davidson, Ramesh Bansal...	<1%